

# WrightPay QA Automation Framework: Living Learning & Reference Guide

**Living Document Notice:** This document serves as the master technical architecture reference, learning textbook, and SDET interview preparation guide for the WrightPay QA Automation Framework ( [qa/automation/](#) ). It is a living document that grows alongside the framework. New test domains and infrastructure capabilities are appended as they are built, preserving all prior documentation and architectural history.

## Table of Contents

- [Part 1 — Overall QA Automation Architecture](#)
- [Part 2 — Playwright & Environment Configuration](#)
  - [File: package.json](#)
  - [File: playwright.config.ts](#)
  - [File: tsconfig.json](#)
  - [File: qa/automation/config/env.config.ts](#)
- [Part 3 — Database Infrastructure](#)
  - [File: qa/automation/database/db-client.ts](#)
- [Part 4 — Redis Infrastructure](#)
  - [File: qa/automation/redis/redis-client.ts](#)
- [Part 5 — BullMQ / Queue Infrastructure](#)
  - [File: qa/automation/queues/queue-client.ts](#)
- [Part 6 — API Client Architecture](#)
  - [File: qa/automation/api/base.api.ts](#)
  - [File: qa/automation/api/auth.api.ts](#)
  - [File: qa/automation/api/users.api.ts](#)
  - [File: qa/automation/api/wallet.api.ts](#)
  - [File: qa/automation/api/cards.api.ts](#)
  - [File: qa/automation/api/beneficiaries.api.ts](#)
  - [File: qa/automation/api/transfers.api.ts](#)
  - [File: qa/automation/api/transactions.api.ts](#)
  - [File: qa/automation/api/exchange-rates.api.ts](#)
  - [File: qa/automation/api/index.ts](#)
- [Part 7 — TypeScript Types](#)
  - [File: qa/automation/api/types.ts](#)
- [Part 8 — Authentication & Playwright Fixtures](#)
  - [File: qa/automation/test-data/user.factory.ts](#)
  - [File: qa/automation/fixtures/api.fixtures.ts](#)
- [Part 9 — Infrastructure Smoke Tests](#)
  - [File: qa/automation/tests/infra-smoke.spec.ts](#)

- [Part 10 — API Smoke Tests](#)
    - [File: qa/automation/tests/api-smoke.spec.ts](#)
    - [File: qa/automation/tests/proof-of-life.spec.ts](#)
  - [Part 11 — Authentication Test Suite](#)
    - [File: qa/automation/tests/auth/auth.spec.ts](#)
  - [Part 12 — Users Test Suite](#)
    - [File: qa/automation/tests/users/users.spec.ts](#)
  - [Part 13 — Wallet Test Suite](#)
    - [File: qa/automation/tests/wallet/wallet.spec.ts](#)
  - [Part 14 — Cards Test Suite](#)
    - [File: qa/automation/tests/cards/cards.spec.ts](#)
  - [Part 15 — Test Data & Test Isolation](#)
  - [Part 16 — Backend Behavior Discovered During Development](#)
  - [Part 17 — Problems Encountered During Development & Solutions](#)
  - [Part 18 — Complete Execution Walkthrough](#)
  - [Part 19 — Important SDET Concepts](#)
  - [Part 20 — API Testing vs. Integration Testing](#)
  - [Part 21 — SDET Interview Preparation](#)
  - [Part 22 — Debugging Guide](#)
  - [Part 23 — Current Test Inventory](#)
  - [Part 24 — Complete QA Automation File Index](#)
  - [Part 25 — Current Progress](#)
  - [Part 26 — Future Documentation Sections](#)
- 

## Part 1 — Overall QA Automation Architecture

---

The WrightPay QA automation framework is an end-to-end, multi-layered test platform located entirely inside `qa/automation/`. It is architected for testing the WrightPay financial payment platform, which consists of a NestJS backend (listening on port 3001), a Next.js frontend (port 3000), a PostgreSQL 15 relational database (port 5432), and a Redis 7 instance powering in-memory operations and BullMQ asynchronous queues (port 6379).

## Current Directory Structure

```

qa/automation/
├── .env.example           # Sample environment variables for database, redis, and API URLs
├── .gitignore            # Git exclusions for node_modules, reports, artifacts, and local .env
├── package.json          # NPM dependencies, Playwright runners, TypeScript engines, and scripts
├── package-lock.json     # Pinned dependency lockfile
├── playwright.config.ts  # Master Playwright test runner configuration
├── tsconfig.json         # TypeScript compiler settings and module path aliases (@api, @fixtures, etc.)
├──
├── config/
│   └── env.config.ts     # Environment loader (dotenv), type-safe config object, and defaults
├── database/
│   └── db-client.ts      # Reusable PostgreSQL client pool with self-healing connection lifecycle
├── redis/
│   └── redis-client.ts   # Reusable ioredis client wrapper for key-value, TTL, and cache assertions
├── queues/
│   └── queue-client.ts   # Reusable BullMQ client for inspecting and polling the 'transfers' queue
├── api/
│   ├── base.api.ts      # Abstract HTTP client wrapping Playwright's APIRequestContext
│   ├── types.ts         # TypeScript DTOs, payload interfaces, enums, and request/response types
│   ├── auth.api.ts      # Domain client for /auth (signup, login, verify-email, logout)
│   ├── users.api.ts     # Domain client for /users (getMe, updateMe)
│   ├── wallet.api.ts    # Domain client for /wallets (getMyWallet)
│   ├── cards.api.ts     # Domain client for /cards (CRUD, freeze, unfreeze, deactivate)
│   ├── beneficiaries.api.ts # Domain client for /beneficiaries
│   ├── transfers.api.ts # Domain client for /transfers (with idempotency support)
│   ├── transactions.api.ts # Domain client for /transactions
│   ├── exchange-rates.api.ts # Domain client for /exchange-rates (rates & quotes)
│   └── index.ts         # Public barrel export for all API clients and models
├── test-data/
│   └── user.factory.ts   # Isolated, deterministic user generation factory
├── fixtures/
│   └── api.fixtures.ts  # Playwright custom test fixture providing pre-authenticated sessions & clients
├── tests/
│   ├── proof-of-life.spec.ts # Sanity check confirming Playwright test runner execution
│   ├── infra-smoke.spec.ts  # Smoke tests verifying PostgreSQL, Redis, and BullMQ connectivity
│   ├── api-smoke.spec.ts    # Smoke tests verifying unauthenticated & authenticated API layers
│   ├── auth/
│   │   └── auth.spec.ts     # 19 tests covering authentication, tokens, hashing, and boundary checks
│   ├── users/
│   │   └── users.spec.ts    # 12 tests covering profile retrieval, PATCH, IDOR, and input validation
│   ├── wallet/
│   │   └── wallet.spec.ts   # 8 tests covering wallet contract, precision, DB parity, equivalents, and idempotency
│   ├── cards/
│   │   └── cards.spec.ts    # 19 tests covering card provisioning, state machine, hard delete, and IDOR
│   ├── api/              # (Placeholder directories for future domain migrations)
│   ├── concurrency/      # (Placeholder for future race condition and concurrent transfer testing)
│   ├── database/         # (Placeholder for future direct database assertion suites)
│   ├── e2e/              # (Placeholder for future cross-domain multi-step scenarios)
│   ├── integration/      # (Placeholder for future async transfer queue integration suites)
│   ├── performance/      # (Placeholder for future latency and throughput benchmarks)
│   └── security/         # (Placeholder for future advanced vulnerability and injection tests)
├── pages/                # Placeholder for future Page Object Models (UI automation)
├── reports/              # Generated HTML reports, JSON outputs, and trace artifacts
└── utils/                # Shared QA utilities

```

## Layer Responsibilities

1. **Configuration Layer ( `config/` )**: Centralizes environment variables, default connection strings, test timeouts, and retry policies. It shields test suites from reading `process.env` directly and ensures type safety across environments ( `local` , `test` , `staging` , `production` ).

- Infrastructure Layer ( `database/` , `redis/` , `queues/` )**: Provides direct access to backend storage and messaging tiers (PostgreSQL, Redis, BullMQ). This enables **cross-layer testing**: testing that an HTTP operation didn't just return HTTP 200, but actually updated the relational database, populated the cache with the right TTL, or queued an asynchronous worker job.
- API Client Layer ( `api/` )**: Encapsulates all HTTP request construction, URL path resolution, headers, serialization, and deserialization behind strongly-typed TypeScript classes. Tests interact with clean domain methods like `cardsApi.freezeCard(id)` rather than constructing raw HTTP requests.
- Test Data Layer ( `test-data/` )**: Contains data factories like `user.factory.ts` that generate collision-free, randomized-yet-deterministic test entities ( `qa_1726645800_1234_1@wrightpay-qa.test` ). This ensures complete test isolation during parallel execution.
- Fixture Layer ( `fixtures/` )**: Extends Playwright's native test runner via Dependency Injection. It automatically orchestrates complex prerequisite flows—such as signing up a fresh user, confirming their email OTP, logging in, acquiring a JWT, configuring a dedicated `APIRequestContext` with `Authorization: Bearer <token>`, and instantiating domain clients—before a test case even begins.
- Test Suite Layer ( `tests/` )**: Contains the executable specifications ( `*.spec.ts` ). Tests declare their dependencies via fixtures, invoke domain API methods, verify HTTP response status and payloads, and execute cross-layer verification against PostgreSQL or Redis.

## Architectural Separation Rationale

- Maintainability & DRY**: If the backend endpoint changes from `/wallets/me` to `/wallet/current`, only `WalletApi.getMyWallet()` is updated in one place. Zero test files need modification.
- Separation of Concerns**: Test cases focus strictly on *business assertions* and *invariants*. They are not polluted by HTTP client boilerplate, header wiring, or database connection pools.
- Safety in Parallel Execution**: Centralizing database pools and test data factories prevents race conditions, deadlocks, and cross-worker interference when running with multiple parallel workers.

## Actual Execution Flows in the Framework

The framework executes two primary styles of test flows depending on the test domain:

### 1. Authenticated Domain Flow (e.g., Cards, Users, Wallet)

Test Runner (Playwright Worker)

- Requests { `authUser` } fixture
  - UserFactory generates unique email & credentials
  - AuthApi calls POST `/auth/signup` (NestJS) → PostgreSQL users record created
  - AuthApi calls POST `/auth/verify-email` (NestJS) → status transitions to 'active'
  - AuthApi calls POST `/auth/login` (NestJS) → JWT `access_token` generated
  - Creates new `APIRequestContext` with `Authorization: Bearer <token>`
- Test executes domain action
  - `authUser.api.cards.createCard(cardData)`
  - BaseApi strips leading slash → POST `http://localhost:3001/api/v1/cards`
  - NestJS `CardsController` → `CardsService` → PostgreSQL `cards` table
- Assertions & Cross-Layer Validation
  - Assert HTTP response status (201 Created)
  - Assert API response payload (masked PAN, UUID, status: 'active')
  - Direct DbClient query: `SELECT * FROM cards WHERE id = $1`
    - Assert exact row values in PostgreSQL
- Fixture Teardown
  - `authUser.authContext.dispose()` closes network sockets

## 2. Infrastructure Smoke Flow (e.g., Infra Smoke)

Test Runner (Playwright Worker)

- └─ 1. Directly invokes dbClient.healthCheck() -> runs 'SELECT 1 as alive' on PostgreSQL
- └─ 2. Directly invokes redisClient.healthCheck() -> sends 'PING', asserts 'PONG'
- └─ 3. Directly invokes transferQueueClient.healthCheck() -> checks BullMQ 'transfers' queue
- └─ 4. Validates connectivity without touching HTTP or mutating production data

# Part 2 — Playwright & Environment Configuration

## File: package.json

### Purpose

Defines the npm project configuration, dependencies, devDependencies, and test execution scripts for the QA automation package.

### Why does this file exist?

The QA framework is housed as an independent Node.js project within `qa/automation/`. This keeps automation dependencies isolated from the NestJS backend and Next.js frontend, avoiding package version conflicts.

### Actual Current Code

```
{
  "name": "wrightpay-qa-automation",
  "version": "1.0.0",
  "private": true,
  "description": "Playwright Test automation framework for WrightPay",
  "scripts": {
    "test": "playwright test",
    "test:headed": "playwright test --headed",
    "test:ui": "playwright test --ui",
    "test:report": "playwright show-report reports/html-report",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "bullmq": "^5.41.0",
    "ioredis": "^5.5.0",
    "pg": "^8.13.3"
  },
  "devDependencies": {
    "@playwright/test": "^1.50.1",
    "@types/node": "^22.13.4",
    "@types/pg": "^8.11.11",
    "dotenv": "^16.4.7",
    "typescript": "^5.7.3"
  }
}
```

### Code Explanation

- `scripts.test`: Runs all Playwright tests in headless mode across all discovered spec files.
- `scripts.typecheck`: Runs `tsc --noEmit` to validate complete TypeScript compilation without producing JavaScript artifacts.
- `dependencies`:

- `bullmq` ( `^5.41.0` ): BullMQ queue consumer/inspector for testing background asynchronous transfer processing.
  - `ioredis` ( `^5.5.0` ): High-performance Redis client for cache verification, key inspection, and BullMQ transport.
  - `pg` ( `^8.13.3` ): Pure JavaScript PostgreSQL client pool for querying database state during cross-layer assertions.
  - **devDependencies** :
    - `@playwright/test` ( `^1.50.1` ): The primary test runner, assertion library, and HTTP client ( `APIRequestContext` ).
    - `dotenv` ( `^16.4.7` ): Environment file reader.
    - `typescript` ( `^5.7.3` ): Compiler providing strict compile-time type checking.
- 

## File: playwright.config.ts

---

### Purpose

The master configuration file that controls test discovery, timeouts, concurrency, reporting, retries, and default HTTP context settings.

### Why does this file exist?

Playwright requires a configuration root to orchestrate worker threads, reporters, test match patterns, and base network options.

## Actual Current Code

```
import { defineConfig, devices } from '@playwright/test';
import { config } from './config/env.config';

/**
 * See https://playwright.dev/docs/test-configuration.
 */
export default defineConfig({
  testDir: './tests',
  /* Maximum time one test can run for. */
  timeout: config.timeout,
  expect: {
    /**
     * Maximum time expect() should wait for the condition to be met.
     */
    timeout: 5000,
  },
  /* Run tests in files in parallel */
  fullyParallel: true,
  /* Fail the build on CI if you accidentally left test.only in the source code. */
  forbidOnly: !!process.env.CI,
  /* Retry on CI only */
  retries: config.retries,
  /* Opt out of parallel tests on CI if needed. */
  workers: config.workers,
  /* Output directory for test artifacts (traces, screenshots, videos) */
  outputDir: './reports/test-artifacts',
  /* Reporter to use. See https://playwright.dev/docs/test-reporters */
  reporter: [
    ['list'],
    ['html', { outputFolder: 'reports/html-report', open: 'never' }],
    ['json', { outputFile: 'reports/test-results.json' }],
  ],
  /* Shared settings for all the projects below. See https://playwright.dev/docs/api/class-testoptions. */
  use: {
    /* Base URL to use in actions like `await page.goto('/')`. */
    baseUrl: config.baseUrl,

    /* Collect trace when retrying the failed test. See https://playwright.dev/docs/trace-viewer */
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    headless: config.headless,
  },

  /* Configure projects for major browsers */
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
  ],
});
```

## Code Explanation & Configuration Impact

- testDir: './tests'**: Instructs Playwright to scan the `./tests` directory for any test files matching default glob patterns (`*.spec.ts` or `*.test.ts`).
- timeout: config.timeout**: Sets the global test timeout (30,000ms by default). If a test hangs due to an unresponsive backend, it is terminated after 30s.
- expect.timeout: 5000**: Maximum time an asynchronous assertion (`expect.poll()` or matcher) will wait before failing.
- fullyParallel: true**: Runs all test files concurrently across available CPU workers.
- forbidOnly: !!process.env.CI**: Fails the test run on continuous integration if a developer accidentally committed `test.only()`.

- **retries: config.retries** : Set to `0` locally for rapid failure feedback, and `2` in CI to guard against transient network glitches.
  - **workers: config.workers** : Caps workers at `2` in CI environments to prevent overwhelming container resources, while defaulting to undefined locally (Playwright utilizes 50-100% of logical CPU cores).
  - **reporter** : Configures multi-reporting:
    - **list** : Real-time streaming command-line output.
    - **html** : Standalone interactive HTML report in `reports/html-report`.
    - **json** : Machine-readable results in `reports/test-results.json` for CI dashboard ingestion.
  - **use.baseURL: config.baseURL** : Sets default URL ( `http://localhost:3000` ). Note: As explained below, API tests use `config.apiBaseUrl` ( `http://localhost:3001/api/v1` ).
- 

## File: tsconfig.json

---

### Purpose

Configures the TypeScript compiler ( `tsc` ), path mapping aliases, and module resolution rules for the QA codebase.

### Why does this file exist?

Allows modern ES2022 syntax, enforces strict type checking, and provides clean import paths like `@api/cards.api` instead of brittle relative paths like `../../api/cards.api`.

## Actual Current Code

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "lib": ["ES2022", "DOM"],
    "baseUrl": ".",
    "paths": {
      "@config/*": ["config/*"],
      "@fixtures/*": ["fixtures/*"],
      "@pages/*": ["pages/*"],
      "@api/*": ["api/*"],
      "@database/*": ["database/*"],
      "@redis/*": ["redis/*"],
      "@queues/*": ["queues/*"],
      "@test-data/*": ["test-data/*"],
      "@utils/*": ["utils/*"]
    },
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true
  },
  "include": [
    "playwright.config.ts",
    "config/**/*.ts",
    "fixtures/**/*.ts",
    "pages/**/*.ts",
    "api/**/*.ts",
    "database/**/*.ts",
    "redis/**/*.ts",
    "queues/**/*.ts",
    "test-data/**/*.ts",
    "tests/**/*.ts",
    "utils/**/*.ts"
  ]
}
```

## Code Explanation

- **target: ES2022** & **module: NodeNext** : Emits modern ECMAScript features supported natively by Node.js 18+.
- **paths** : Configures path aliases for each major layer ( **@api/\*** , **@fixtures/\*** , **@database/\*** , etc.).
- **strict: true** , **noImplicitAny: true** , **strictNullChecks: true** : Guarantees type safety across the framework, catching null pointer exceptions, unhandled undefined values, and typing mistakes at compile time.

## File: qa/automation/config/env.config.ts

### Purpose

Loads environment variables from **.env** , validates them, applies fallback defaults, and exports a frozen, typed **config** object.

### Why does this file exist?

Eliminates scattered **process.env** calls throughout test files. If a configuration key changes, it is updated in this single file.

## Actual Current Code

```
import * as path from 'path';
import * as dotenv from 'dotenv';

// Load environment variables from .env file if present
dotenv.config({ path: path.resolve(__dirname, '../.env') });

export type EnvironmentType = 'local' | 'test' | 'staging' | 'production';

export interface TestConfig {
  env: EnvironmentType;
  baseUrl: string;
  apiBaseUrl: string;
  databaseUrl: string;
  redisUrl: string;
  headless: boolean;
  timeout: number;
  retries: number;
  workers: number | undefined;
}

const getEnv = (key: string, defaultValue: string = ''): string => {
  return process.env[key] || defaultValue;
};

const currentEnv = (getEnv('TEST_ENV', 'local').toLowerCase() as EnvironmentType);

export const config: TestConfig = {
  env: currentEnv,
  baseUrl: getEnv('BASE_URL', 'http://localhost:3000'),
  apiBaseUrl: getEnv('API_BASE_URL', 'http://localhost:3001/api/v1'),
  databaseUrl: getEnv('DATABASE_URL', 'postgresql://postgres:password@localhost:5432/wrightpay'),
  redisUrl: getEnv('REDIS_URL', 'redis://localhost:6379'),
  headless: getEnv('HEADLESS', 'true') !== 'false',
  timeout: parseInt(getEnv('TEST_TIMEOUT', '30000'), 10),
  retries: parseInt(getEnv('TEST_RETRIES', currentEnv === 'local' ? '0' : '2'), 10),
  workers: process.env.CI ? 2 : undefined,
};
```

## Code Explanation & Key Distinctions

- **baseUrl** vs **apiBaseUrl** :
  - **baseUrl** ( `http://localhost:3000` ): Points to the Next.js frontend UI web application.
  - **apiBaseUrl** ( `http://localhost:3001/api/v1` ): Points directly to the NestJS backend REST API gateway. Playwright API fixtures and API client instances bind to **apiBaseUrl** .
- **databaseUrl** : Direct connection URI to PostgreSQL. Default: `postgresql://postgres:password@localhost:5432/wrightpay` .
- **redisUrl** : Direct connection URI to Redis. Default: `redis://localhost:6379` .
- **getEnv** helper: Safely inspects `process.env[key]` and returns the fallback if the variable is unset or empty.

## Part 3 — Database Infrastructure

---

### File: qa/automation/database/db-client.ts

---

#### Purpose

Provides a singleton PostgreSQL connection pool manager, safe parameterized query helpers, lightweight health checks, and self-healing connection lifecycle handling.

#### Why does this file exist?

Testing an enterprise payment API cannot rely exclusively on black-box HTTP responses. An API could return `200 OK` while silently failing to write records to disk, failing to hash sensitive credentials, or corrupting account balances. `DbClient` gives QA the power to perform cross-layer assertions against PostgreSQL.

## Actual Current Code

```
import { Pool, QueryResult, QueryResultRow, PoolConfig } from 'pg';
import { config } from '../config/env.config';

export class DbClient {
  private pool!: Pool;
  private connectionString: string;
  private options?: Omit<PoolConfig, 'connectionString'>;

  constructor(connectionString?: string, options?: Omit<PoolConfig, 'connectionString'>) {
    this.connectionString = connectionString || config.databaseUrl;
    this.options = options;
    this.initPool();
  }

  private initPool(): void {
    this.pool = new Pool({
      connectionString: this.connectionString,
      max: 10,
      idleTimeoutMillis: 30000,
      connectionTimeoutMillis: 5000,
      ...this.options,
    });

    this.pool.on('error', (err) => {
      console.error('[DbClient] Unexpected error on idle PostgreSQL client:', err.message);
    });
  }

  private getActivePool(): Pool {
    if ((this.pool as any).ended) {
      this.initPool();
    }
    return this.pool;
  }

  /**
   * Execute a parameterized SQL query safely.
   * Never interpolate values directly into the query string.
   */
  async query<T extends QueryResultRow = any>(
    text: string,
    values?: any[],
  ): Promise<QueryResult<T>> {
    return this.getActivePool().query<T>(text, values);
  }

  /**
   * Convenience helper to execute a query and return the first row, or null if no rows matched.
   */
  async queryOne<T extends QueryResultRow = any>(
    text: string,
    values?: any[],
  ): Promise<T | null> {
    const result = await this.query<T>(text, values);
    return result.rows[0] || null;
  }

  /**
   * Perform a lightweight health check to confirm PostgreSQL connectivity.
   */
  async healthCheck(): Promise<boolean> {
    try {
      const result = await this.query('SELECT 1 as alive');
      return result.rows.length > 0 && result.rows[0].alive === 1;
    } catch (error) {
      console.error('[DbClient] Health check failed:', error instanceof Error ? error.message : error);
      return false;
    }
  }
}
```

```

    * Gracefully close all connections in the pool.
    */
    async close(): Promise<void> {
        if (this.pool && !(this.pool as any).ended) {
            await this.pool.end();
        }
    }

    /**
     * Access the underlying pg.Pool if needed.
     */
    getPool(): Pool {
        return this.getActivePool();
    }
}

export const dbClient = new DbClient();
export default dbClient;

```

## Code Explanation Block-by-Block

- **Lines 9–13 ( `constructor` )**: Initializes the client using `config.databaseUrl` and invokes `initPool()`.
- **Lines 15–27 ( `initPool` )**: Instantiates `new Pool({...})` with a maximum of 10 concurrent connections per worker, an idle timeout of 30 seconds, and a connection acquisition timeout of 5 seconds. Attaches an error handler to idle clients to prevent uncaught process crashes.
- **Lines 29–34 ( `getActivePool` ) - Self-Healing Lifecycle:**
  - *Engineering Problem Solved*: In Playwright multi-worker parallel execution, if any test file or teardown hook calls `dbClient.close()`, the underlying `pg.Pool` transitions to `ended = true`. Subsequent queries in the same worker would crash with `Error: Cannot use a pool after calling end()`.
  - *Mechanism*: Checks `(this.pool as any).ended`. If ended, it transparently calls `this.initPool()` to re-instantiate the pool on the fly.
- **Lines 40–45 ( `query` )**: Executes parameterized SQL queries using placeholders ( `$1`, `$2` ). Parameterization prevents SQL injection and syntax errors when values contain quotes or special characters.
- **Lines 50–56 ( `queryOne` )**: Convenience wrapper returning `result.rows[0] || null`. Eliminates repetitive `res.rows.length > 0 ? res.rows[0] : null` checks across test suites.
- **Lines 61–69 ( `healthCheck` )**: Runs `SELECT 1 as alive` inside a `try/catch`. Returns `true` if connected, `false` on connection error.
- **Lines 74–78 ( `close` )**: Safe termination guarded by `!(this.pool as any).ended`.

## Why Database Assertions Matter in API Testing

1. **Security & Cryptography**: Black-box APIs never return password hashes. Direct DB verification proves that `passwordHash` starts with `$argon2` and was not stored plaintext.
2. **Default Entity Provisioning**: In WrightPay, `POST /auth/signup` returns only `{ message, userId }`. Querying the database proves that a default `wallets` record in `EUR` with balance `0.00` was initialized.
3. **Hard vs. Soft Deletion Invariants**: When `DELETE /cards/:id` is called, the API returns `{ message: "Card successfully deleted" }`. Querying `SELECT count(*) FROM cards WHERE id = $1` proves the row was physically purged rather than just soft-deleted or hidden.

## Part 4 — Redis Infrastructure

---

### File: qa/automation/redis/redis-client.ts

---

#### Purpose

Provides a singleton Redis client wrapper utilizing `ioredis` for interacting with in-memory stores, asserting key lifecycles, and checking time-to-live (TTL).

#### Why does this file exist?

WrightPay relies on Redis for caching, OTP verification, rate limiting, and BullMQ queue storage. `RedisClient` provides an isolated test interface to inspect and assert on Redis without polluting application logic.

## Actual Current Code

```
import Redis, { RedisOptions } from 'ioredis';
import { config } from '../config/env.config';

export class RedisClient {
  private client: Redis;

  constructor(redisUrl?: string, options?: RedisOptions) {
    const url = redisUrl || config.redisUrl;
    this.client = new Redis(url, {
      lazyConnect: true,
      maxRetriesPerRequest: 3,
      retryStrategy: (times) => {
        if (times > 3) return null;
        return Math.min(times * 100, 1000);
      },
      ...options,
    });

    this.client.on('error', (err) => {
      console.error('[RedisClient] Error encountered on Redis connection:', err.message);
    });
  }

  /**
   * Connect to Redis if not already connected.
   */
  async connect(): Promise<void> {
    if (this.client.status === 'wait' || this.client.status === 'close') {
      await this.client.connect();
    }
  }

  /**
   * Retrieve string value by key. Returns null if not found.
   */
  async get(key: string): Promise<string | null> {
    await this.connect();
    return this.client.get(key);
  }

  /**
   * Set key-value pair with optional TTL in seconds.
   */
  async set(key: string, value: string, ttlSeconds?: number): Promise<'OK' | null> {
    await this.connect();
    if (ttlSeconds && ttlSeconds > 0) {
      return this.client.set(key, value, 'EX', ttlSeconds);
    }
    return this.client.set(key, value);
  }

  /**
   * Delete a specific key. Returns number of keys deleted (0 or 1).
   */
  async del(key: string): Promise<number> {
    await this.connect();
    return this.client.del(key);
  }

  /**
   * Get remaining TTL for a key in seconds.
   * Returns -2 if key does not exist, -1 if key exists with no expiry.
   */
  async ttl(key: string): Promise<number> {
    await this.connect();
    return this.client.ttl(key);
  }

  /**
   * Check if a key exists in Redis.
   */
}
```

```

async exists(key: string): Promise<boolean> {
  await this.connect();
  const count = await this.client.exists(key);
  return count > 0;
}

/**
 * Health check verifying ping/pong response from Redis.
 */
async healthCheck(): Promise<boolean> {
  try {
    await this.connect();
    const response = await this.client.ping();
    return response === 'PONG';
  } catch (error) {
    console.error('[RedisClient] Health check failed:', error instanceof Error ? error.message : error);
    return false;
  }
}

/**
 * Graceful disconnect from Redis.
 */
async close(): Promise<void> {
  if (this.client.status === 'ready' || this.client.status === 'connecting') {
    await this.client.quit();
  }
}

/**
 * Get underlying IORedis instance for advanced operations.
 */
getClient(): Redis {
  return this.client;
}
}

export const redisClient = new RedisClient();
export default redisClient;

```

## Code Explanation Block-by-Block

- **Lines 8–17 ( `constructor` )**: Configures `ioredis` with `lazyConnect: true` so the connection is not opened until a command is issued. Configures a capped retry strategy (up to 3 attempts, backoff between 100ms and 1000ms) to prevent tests from blocking indefinitely if Redis is down.
- **Lines 27–31 ( `connect` )**: Ensures the client is in a connected state before executing commands.
- **Lines 44–50 ( `set` )**: Implements key setting with optional `'EX'` (expire in seconds).
- **Lines 64–67 ( `tTL` )**: Returns seconds until key expiration. Allows asserting that ephemeral tokens or OTPs expire as designed.
- **Lines 81–90 ( `healthCheck` )**: Sends a `PING` command and asserts that Redis answers with `PONG`.

## Part 5 — BullMQ / Queue Infrastructure

### File: qa/automation/queues/queue-client.ts

#### Purpose

Provides a client to inspect, poll, and verify the BullMQ `'transfers'` queue and its `'process-transfer'` jobs.

## Why does this file exist?

In WrightPay, money transfers are processed asynchronously. When `POST /transfers` is called, the API immediately creates a transaction in status `PENDING` and enqueues a background job rather than executing the transfer synchronously.

`TransferQueueClient` enables QA to verify that jobs are enqueued with correct payloads and to poll job completion.

## Actual Current Code

```
import { Queue, Job } from 'bullmq';
import IORedis from 'ioredis';
import { config } from '../config/env.config';

export const TRANSFERS_QUEUE_NAME = 'transfers';
export const PROCESS_TRANSFER_JOB_NAME = 'process-transfer';

export interface TransferJobData {
  transactionId: string;
}

export class TransferQueueClient {
  private queue: Queue<TransferJobData>;
  private connection: IORedis;

  constructor(queueName: string = TRANSFERS_QUEUE_NAME, redisUrl?: string) {
    const url = redisUrl || config.redisUrl;

    // BullMQ requires maxRetriesPerRequest: null for blocking Redis operations
    this.connection = new IORedis(url, {
      maxRetriesPerRequest: null,
      lazyConnect: true,
    });

    this.connection.on('error', (err) => {
      console.error('[TransferQueueClient] Redis connection error:', err.message);
    });

    this.queue = new Queue<TransferJobData>(queueName, {
      connection: this.connection,
    });
  }

  /**
   * Fetch a job by its unique jobId (e.g., "transfer-<transactionId>").
   */
  async getJob(jobId: string): Promise<Job<TransferJobData> | undefined> {
    return this.queue.getJob(jobId);
  }

  /**
   * Retrieve the current state of a job ('completed' | 'failed' | 'delayed' | 'active' | 'waiting' | 'unknown').
   */
  async getJobState(jobId: string): Promise<string | undefined> {
    const job = await this.getJob(jobId);
    if (!job) return undefined;
    return job.getState();
  }

  /**
   * Inspect job counts by status (active, completed, failed, delayed, waiting, paused).
   */
  async getJobCounts(): Promise<Record<string, number>> {
    return this.queue.getJobCounts('active', 'completed', 'failed', 'delayed', 'waiting', 'paused');
  }

  /**
   * Poll and wait for a job to reach an expected status within a timeout window.
   * Useful in async integration tests where transfers process in background.
   */
  async waitForJobStatus(
    jobId: string,
    targetStatus: string | string[],
    timeoutMs: number = 10000,
    pollIntervalMs: number = 200,
  ): Promise<Job<TransferJobData> | undefined> {
    const targets = Array.isArray(targetStatus) ? targetStatus : [targetStatus];
    const startTime = Date.now();

    while (Date.now() - startTime < timeoutMs) {
      const job = await this.getJob(jobId);
    }
  }
}
```

```

    if (job) {
      const currentState = await job.getState();
      if (targets.includes(currentState)) {
        return job;
      }
    }
    await new Promise((resolve) => setTimeout(resolve, pollIntervalMs));
  }

  const finalJob = await this.getJob(jobId);
  const finalState = finalJob ? await finalJob.getState() : 'not_found';
  throw new Error(
    `Job ${jobId} did not reach status [${targets.join(', ')}] within ${timeoutMs}ms. Current state: ${finalState}
  `);
};

/**
 * Health check confirming that BullMQ can inspect the queue on Redis without mutating it.
 */
async healthCheck(): Promise<boolean> {
  try {
    const counts = await this.getJobCounts();
    return counts !== null && typeof counts === 'object';
  } catch (error) {
    console.error('[TransferQueueClient] Health check failed:', error instanceof Error ? error.message : error);
    return false;
  }
}

/**
 * Graceful cleanup of BullMQ queue and its dedicated Redis connection.
 */
async close(): Promise<void> {
  try {
    await this.queue.close();
  } finally {
    if (this.connection.status === 'ready' || this.connection.status === 'connecting') {
      await this.connection.quit();
    }
  }
}

/**
 * Access the underlying BullMQ Queue instance.
 */
getQueue(): Queue<TransferJobData> {
  return this.queue;
}
}

export const transferQueueClient = new TransferQueueClient();
export default transferQueueClient;

```

## Code Explanation Block-by-Block

- **Lines 5–10:** Defines constants verified from the backend codebase: `TRANSFERS_QUEUE_NAME = 'transfers'`, `PROCESS_TRANSFER_JOB_NAME = 'process-transfer'`, and `TransferJobData { transactionId: string }`.
- **Lines 19–23:** BullMQ requires `maxRetriesPerRequest: null` on its Redis connection because BullMQ uses blocking Redis operations ( `BRPOPLPUSH`, `BLMOVE` ). If retries are enabled, `ioredis` would prematurely reject commands during blocking waits.
- **Lines 37–48 ( `getJob` & `getJobState` ):** Retrieves BullMQ job instances and states ( `active`, `waiting`, `completed`, `failed` ).
- **Lines 61–86 ( `waitForJobStatus` ):** Deterministic polling loop with configurable interval (default 200ms) and timeout (default 10s). Avoids arbitrary `sleep()` statements in asynchronous transfer testing.
- **Lines 104–112 ( `close` ):** Cleanly closes both the BullMQ queue listeners and the dedicated Redis socket connection.

## Part 6 — API Client Architecture

### Architectural Distinction: API Client vs. Test vs. Fixture

| Layer                                                    | Responsibility                                                                                                                                | What It Does NOT Do                                                                    |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>API Client</b> ( <code>api/*.api.ts</code> )          | Pure HTTP transport abstraction. Constructs URLs, serializes bodies, applies query params, sends requests. Returns <code>APIResponse</code> . | Does NOT assert status codes or response bodies. Has zero <code>expect()</code> calls. |
| <b>Fixture</b> ( <code>fixtures/api.fixtures.ts</code> ) | Lifecycle orchestration & dependency injection. Sets up users, logins, tokens, contexts, and tears down sockets.                              | Does NOT contain domain business logic or specific test assertions.                    |
| <b>Test</b> ( <code>tests/**/*.spec.ts</code> )          | Business specification. Declares fixtures, calls client methods, asserts HTTP status, asserts JSON contracts, asserts DB state.               | Does NOT manually build HTTP requests, format headers, or parse raw URLs.              |
| <b>APIRequestContext</b> (Playwright)                    | Low-level browser-grade HTTP client with cookie jars, redirects, and connection pooling.                                                      | Does NOT know anything about WrightPay domains or business endpoints.                  |

### Why Assertions Belong in Tests Rather than API Clients

If an API client had `expect(response.status()).toBe(200)` hardcoded inside `cardsApi.createCard()`, it would be impossible to use that same client method to test negative scenarios (e.g., asserting that an invalid card returns `400 Bad Request` or an unauthenticated call returns `401 Unauthorized`). API clients must remain completely neutral and return the raw `APIResponse`.

### File: `qa/automation/api/base.api.ts`

#### Purpose

Abstract base class wrapping Playwright's `APIRequestContext` to provide standard HTTP methods ( `get`, `post`, `patch`, `delete` ) and path normalization.

#### Why does this file exist?

Normalizes request dispatching across all domain clients and resolves URL pathing bugs.

## Actual Current Code

```
import { APIRequestContext, APIResponse } from '@playwright/test';

export interface RequestOptions {
  headers?: Record<string, string>;
  params?: Record<string, any>;
  data?: any;
}

export abstract class BaseApi {
  constructor(protected request: APIRequestContext) {}

  /**
   * Normalizes path to prevent leading slashes from stripping baseUrl subpaths like /api/v1/.
   */
  protected resolvePath(path: string): string {
    return path.startsWith('/') ? path.slice(1) : path;
  }

  protected async get(path: string, options?: RequestOptions): Promise<APIResponse> {
    return this.request.get(this.resolvePath(path), {
      headers: options?.headers,
      params: options?.params,
    });
  }

  protected async post(path: string, options?: RequestOptions): Promise<APIResponse> {
    return this.request.post(this.resolvePath(path), {
      headers: options?.headers,
      params: options?.params,
      data: options?.data,
    });
  }

  protected async patch(path: string, options?: RequestOptions): Promise<APIResponse> {
    return this.request.patch(this.resolvePath(path), {
      headers: options?.headers,
      params: options?.params,
      data: options?.data,
    });
  }

  protected async delete(path: string, options?: RequestOptions): Promise<APIResponse> {
    return this.request.delete(this.resolvePath(path), {
      headers: options?.headers,
      params: options?.params,
    });
  }
}
```

## Code Explanation: The Critical `resolvePath` Method

In Node.js standard URL resolution and Playwright's underlying request client:

- If `baseUrl` is `http://localhost:3001/api/v1`
- Resolving  `'/auth/login'`  (with a leading slash) resolves against the **origin**, stripping  `/api/v1`  and making the request to  `http://localhost:3001/auth/login`  (404 Not Found).
- Resolving  `'auth/login'`  (without leading slash) resolves relative to the path, correctly resulting in  `http://localhost:3001/api/v1/auth/login` .
- `resolvePath(path)`  checks  `path.startsWith('/') ? path.slice(1) : path` , ensuring that whether a developer passes  `'/cards'`  or  `'cards'` , the URL always resolves correctly.

## File: qa/automation/api/auth.api.ts

### Purpose

Domain API client for authentication endpoints ( `/auth/*` ).

### Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from './base.api';
import { SignupRequest, LoginRequest, VerifyEmailRequest } from './types';

export class AuthApi extends BaseApi {
  async signup(data: SignupRequest, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post('auth/signup', { data, headers });
  }

  async login(data: LoginRequest, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post('auth/login', { data, headers });
  }

  async verifyEmail(data: VerifyEmailRequest, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post('auth/verify-email', { data, headers });
  }

  async logout(headers?: Record<string, string>): Promise<APIResponse> {
    return this.post('auth/logout', { headers });
  }
}
```

### Methods Explained

- **signup(data, headers)** : `POST /auth/signup` . Sends `firstName` , `lastName` , `email` , `password` , `agreeTerms` .
- **login(data, headers)** : `POST /auth/login` . Sends `email` , `password` . Returns `{ access_token, user }` .
- **verifyEmail(data, headers)** : `POST /auth/verify-email` . Sends `email` , `code` . Transitions user to `active` .
- **logout(headers)** : `POST /auth/logout` . Invalidates/acknowledges logout for the caller.

## File: qa/automation/api/users.api.ts

### Purpose

Domain API client for user profile management ( `/users/*` ).

### Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from './base.api';
import { UpdateUserRequest } from './types';

export class UsersApi extends BaseApi {
  async getMe(headers?: Record<string, string>): Promise<APIResponse> {
    return this.get('users/me', { headers });
  }

  async updateMe(data: UpdateUserRequest, headers?: Record<string, string>): Promise<APIResponse> {
    return this.patch('users/me', { data, headers });
  }
}
```

## Methods Explained

- **getMe(headers)** : `GET /users/me` . Fetches authenticated user's profile based on the Bearer token.
  - **updateMe(data, headers)** : `PATCH /users/me` . Updates `name` , `countryOfResidence` , or `defaultCurrency` .
- 

## File: qa/automation/api/wallet.api.ts

---

### Purpose

Domain API client for the authenticated user's wallet ( `/wallets/*` ).

### Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from './base.api';

export class WalletApi extends BaseApi {
  async getMyWallet(headers?: Record<string, string>): Promise<APIResponse> {
    return this.get('wallets/me', { headers });
  }
}
```

## Methods Explained

- **getMyWallet(headers)** : `GET /wallets/me` . Retrieves the authenticated user's primary wallet including base balance and calculated multi-currency equivalents.
- 

## File: qa/automation/api/cards.api.ts

---

### Purpose

Domain API client for virtual/physical payment card lifecycle management ( `/cards/*` ).

## Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from './base.api';
import { CreateCardRequest } from './types';

export class CardsApi extends BaseApi {
  async getMyCards(headers?: Record<string, string>): Promise<APIResponse> {
    return this.get('cards', { headers });
  }

  async createCard(data: CreateCardRequest, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post('cards', { data, headers });
  }

  async freezeCard(id: string, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post(`cards/${id}/freeze`, { headers });
  }

  async unfreezeCard(id: string, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post(`cards/${id}/unfreeze`, { headers });
  }

  async deactivateCard(id: string, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post(`cards/${id}/deactivate`, { headers });
  }

  async deleteCard(id: string, headers?: Record<string, string>): Promise<APIResponse> {
    return this.delete(`cards/${id}`, { headers });
  }
}
```

## Methods Explained

- getMyCards(headers)** : **GET /cards** . Lists all cards owned by the authenticated caller.
- createCard(data, headers)** : **POST /cards** . Provisions a card ( **cardholderName** , **cardNumber** , **expiryDate** , **cvv** , **type** ).
- freezeCard(id, headers)** : **POST /cards/:id/freeze** . Transitions card status from **active** to **frozen** .
- unfreezeCard(id, headers)** : **POST /cards/:id/unfreeze** . Transitions card status from **frozen** to **active** .
- deactivateCard(id, headers)** : **POST /cards/:id/deactivate** . Transitions card to permanent terminal status **deactivated** .
- deleteCard(id, headers)** : **DELETE /cards/:id** . Permanently deletes the card from PostgreSQL.

---

## File: qa/automation/api/beneficiaries.api.ts

---

### Purpose

Domain API client for recipient management ( **/beneficiaries/\*** ).

## Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from './base.api';
import { CreateBeneficiaryRequest } from './types';

export class BeneficiariesApi extends BaseApi {
  async getMyBeneficiaries(headers?: Record<string, string>): Promise<APIResponse> {
    return this.get('beneficiaries', { headers });
  }

  async createBeneficiary(data: CreateBeneficiaryRequest, headers?: Record<string, string>): Promise<APIResponse> {
    return this.post('beneficiaries', { data, headers });
  }

  async deleteBeneficiary(id: string, headers?: Record<string, string>): Promise<APIResponse> {
    return this.delete(`beneficiaries/${id}`, { headers });
  }
}
```

## File: qa/automation/api/transfers.api.ts

### Purpose

Domain API client for initiating money transfers ( `/transfers/*` ).

## Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from './base.api';
import { CreateTransferRequest } from './types';

export class TransfersApi extends BaseApi {
  /**
   * Initiate a transfer.
   * Callers must explicitly supply the idempotencyKey so tests can deliberately test duplicate keys.
   */
  async createTransfer(
    data: CreateTransferRequest,
    idempotencyKey?: string,
    headers?: Record<string, string>,
  ): Promise<APIResponse> {
    const mergedHeaders: Record<string, string> = { ...headers };
    if (idempotencyKey !== undefined) {
      mergedHeaders['Idempotency-Key'] = idempotencyKey;
    }
    return this.post('transfers', { data, headers: mergedHeaders });
  }
}
```

### Key Design Feature: Idempotency Key Handling

Transfers support an `Idempotency-Key` HTTP header. By allowing the test to explicitly provide or omit `idempotencyKey`, tests can verify that retrying with the same idempotency key returns the cached response rather than creating duplicate money transfers.

## File: qa/automation/api/transactions.api.ts

---

### Purpose

Domain API client for listing and retrieving historical transactions ( `/transactions/*` ).

### Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from '../base.api';
import { GetTransactionsQuery } from '../types';

export class TransactionsApi extends BaseApi {
  async getMyTransactions(
    params?: GetTransactionsQuery,
    headers?: Record<string, string>,
  ): Promise<APIResponse> {
    return this.get('transactions', { params, headers });
  }

  async getTransactionById(id: string, headers?: Record<string, string>): Promise<APIResponse> {
    return this.get(`transactions/${id}`, { headers });
  }
}
```

## File: qa/automation/api/exchange-rates.api.ts

---

### Purpose

Domain API client for public exchange rates and currency conversion quotes ( `/exchange-rates/*` ).

### Actual Current Code

```
import { APIResponse } from '@playwright/test';
import { BaseApi } from '../base.api';
import { GetQuoteQuery } from '../types';

export class ExchangeRatesApi extends BaseApi {
  async getAllRates(headers?: Record<string, string>): Promise<APIResponse> {
    return this.get('exchange-rates', { headers });
  }

  async getQuote(params: GetQuoteQuery, headers?: Record<string, string>): Promise<APIResponse> {
    return this.get('exchange-rates/quote', { params, headers });
  }
}
```

## File: qa/automation/api/index.ts

---

### Purpose

Barrel file that consolidates all API clients and type definitions into a single import location.

## Actual Current Code

```
export * from './types';
export * from './base.api';
export * from './auth.api';
export * from './users.api';
export * from './wallet.api';
export * from './cards.api';
export * from './beneficiaries.api';
export * from './transfers.api';
export * from './transactions.api';
export * from './exchange-rates.api';
```

---

## Part 7 — TypeScript Types

---

### File: qa/automation/api/types.ts

---

#### Purpose

Defines all data transfer objects (DTOs), request payloads, enums, and query parameter models for the WrightPay API automation suite.

## Actual Current Code

```
export type Currency = 'EUR' | 'GBP' | 'USD' | 'INR' | 'PLN';

export type CardType = 'debit' | 'credit' | 'DEBIT' | 'CREDIT';

export type CardStatus = 'active' | 'frozen' | 'deactivated' | 'declined' | 'pending';

export type BeneficiaryPayoutMethod = 'BANK_ACCOUNT' | 'UPI';

export type TransactionStatus =
  | 'PENDING'
  | 'PROCESSING'
  | 'COMPLETED'
  | 'FAILED'
  | 'SUSPICIOUS';

export interface SignupRequest {
  firstName: string;
  lastName: string;
  email: string;
  password: string;
  agreeTerms: boolean;
}

export interface LoginRequest {
  email: string;
  password: string;
}

export interface VerifyEmailRequest {
  email: string;
  code: string;
}

export interface UpdateUserRequest {
  name?: string;
  countryOfResidence?: string;
  defaultCurrency?: Currency;
}

export interface CreateCardRequest {
  cardholderName: string;
  type?: CardType;
  cardNumber?: string;
  lastFourDigits?: string;
  expiryDate: string;
  cvv?: string;
}

export interface CreateBeneficiaryRequest {
  name: string;
  currency: Currency;
  payoutMethod?: BeneficiaryPayoutMethod;
  accountNumber?: string;
  bankCode?: string;
  ifscCode?: string;
  upiId?: string;
  bankName?: string;
}

export interface CreateTransferRequest {
  beneficiaryId: string;
  sourceWalletId: string;
  sendAmount: number;
  destinationCurrency: Currency;
}

export interface GetTransactionsQuery {
  status?: TransactionStatus;
  reference?: string;
  limit?: number;
  offset?: number;
```

```
}

export interface GetQuoteQuery {
  from: Currency;
  to: Currency;
  amount: number;
}
```

## TypeScript Compile-Time Typing vs. Runtime API Validation

- **Compile-Time Checking:** Prevents typos during test authoring (e.g., passing `currncy: 'EUR'` instead of `currency: 'EUR'`).
- **Runtime Validation:** TypeScript types **do not exist at runtime**. The fact that `CreateCardRequest` has `cardholderName: string` does not prevent a test from deliberately passing `cardholderName: ""` or `type: "unsupported" as any`. In an SDET framework, we frequently cast invalid types (`as any`) to verify that the backend's runtime NestJS `ValidationPipe` correctly catches and rejects invalid payloads with `400 Bad Request`.

---

# Part 8 — Authentication & Playwright Fixtures

---

## File: `qa/automation/test-data/user.factory.ts`

---

### Purpose

Generates collision-free, isolated, deterministic user credentials for automated test runs.

### Why does this file exist?

If automated tests share hardcoded emails like `test@wrightpay.com`, parallel workers collide, signup fails on duplicate email constraints, and test order dependency is introduced. `UserFactory` generates completely independent test users.

## Actual Current Code

```
import { SignupRequest } from '../api/types';

export interface TestUserData extends SignupRequest {
  fullName: string;
}

let userCounter = 0;

/**
 * Generate unique, deterministic-yet-isolated user signup data for automated testing.
 */
export function generateTestUserData(overrides?: Partial<SignupRequest>): TestUserData {
  userCounter += 1;
  const uniqueId = `${Date.now()}_${process.pid}_${userCounter}_${Math.floor(Math.random() * 10000)}`;
  const firstName = overrides?.firstName || `QAUser${userCounter}`;
  const lastName = overrides?.lastName || `Test`;
  const email = overrides?.email || `qa_${uniqueId}@wrightpay-qa.test`.toLowerCase();
  const password = overrides?.password || 'TestPassword123!';
  const agreeTerms = overrides?.agreeTerms !== undefined ? overrides.agreeTerms : true;

  return {
    firstName,
    lastName,
    fullName: `${firstName} ${lastName}`,
    email,
    password,
    agreeTerms,
  };
}
```

## Code Explanation: The Unique ID Strategy

- `Date.now()` : Millisecond timestamp ensuring monotonic time uniqueness.
- `process.pid` : Process ID of the Node.js / Playwright worker. Distinguishes workers running concurrently on the same machine.
- `userCounter` : Monotonically incrementing integer within the worker process.
- `Math.random()` : Additional randomness barrier to prevent collisions during sub-millisecond iterations.

## File: qa/automation/fixtures/api.fixtures.ts

### Purpose

Playwright custom test fixture extending `@playwright/test` to provide dependency injection for authenticated and unauthenticated contexts, domain API clients, and database/Redis/queue singletons.

### Why does this file exist?

Eliminates hundreds of lines of repetitive authentication setup from every test file. Tests simply declare `{ authUser }` in their test signature, and Playwright provisions a clean, verified user session automatically.

## Actual Current Code

```
import { test as base, APIRequestContext } from '@playwright/test';
import { config } from '../config/env.config';
import {
  AuthApi,
  UsersApi,
  WalletApi,
  CardsApi,
  BeneficiariesApi,
  TransfersApi,
  TransactionsApi,
  ExchangeRatesApi,
} from '../api';
import { generateTestUserData, TestUserData } from '../test-data/user.factory';
import { dbClient, DbClient } from '../database/db-client';
import { redisClient, RedisClient } from '../redis/redis-client';
import { transferQueueClient, TransferQueueClient } from '../queues/queue-client';

export interface AuthenticatedUserSession {
  user: {
    id: string;
    email: string;
    name: string;
    password: string;
  };
  token: string;
  authContext: APIRequestContext;
  api: {
    auth: AuthApi;
    users: UsersApi;
    wallet: WalletApi;
    cards: CardsApi;
    beneficiaries: BeneficiariesApi;
    transfers: TransfersApi;
    transactions: TransactionsApi;
    exchangeRates: ExchangeRatesApi;
  };
}

export interface ApiFixtures {
  /** Unauthenticated API request context pointed at API_BASE_URL */
  apiContext: APIRequestContext;
  /** Unauthenticated AuthApi client */
  authApi: AuthApi;
  /** Unauthenticated ExchangeRatesApi client */
  exchangeRatesApi: ExchangeRatesApi;
  /** Fully provisioned and authenticated user session */
  authUser: AuthenticatedUserSession;
  /** Database test client singleton */
  db: DbClient;
  /** Redis test client singleton */
  redis: RedisClient;
  /** BullMQ transfers queue client singleton */
  queue: TransferQueueClient;
}

const getNormalizedApiBaseUrl = (): string => {
  return config.apiBaseUrl.endsWith('/') ? config.apiBaseUrl : `${config.apiBaseUrl}/`;
};

export const test = base.extend<ApiFixtures>({
  apiContext: async ({ playwright }, use) => {
    const context = await playwright.request.newContext({
      baseURL: getNormalizedApiBaseUrl(),
    });
    await use(context);
    await context.dispose();
  },

  authApi: async ({ apiContext }, use) => {
    await use(new AuthApi(apiContext));
  },
});
```

```

exchangeRatesApi: async ({ apiContext }, use) => {
  await use(new ExchangeRatesApi(apiContext));
},

authUser: async ({ playwright, apiContext }, use) => {
  const authApi = new AuthApi(apiContext);
  const testUser = generateTestUserData();

  // 1. Real API Signup
  const signupRes = await authApi.signup(testUser);
  if (!signupRes.ok()) {
    const errorText = await signupRes.text();
    throw new Error(`[AuthFixture] Signup failed for ${testUser.email} with status ${signupRes.status()}: ${errorText}`);
  }
  const signupData = await signupRes.json();
  const userId = signupData.userId;

  // 2. Real API Email Verification
  // Default development verification code is 123456
  let verifyRes = await authApi.verifyEmail({
    email: testUser.email,
    code: '123456',
  });

  if (!verifyRes.ok()) {
    // If code 123456 failed (e.g. In non-dev environment), retrieve generated OTP from database
    const row = await dbClient.queryOne<{ verificationCode?: string; verification_code?: string }>(`
      SELECT "verificationCode" FROM email_verifications WHERE email = $1 ORDER BY "createdAt" DESC LIMIT 1`,
      [testUser.email],
    );
    const dbCode = row?.verificationCode || row?.verification_code;
    if (dbCode) {
      verifyRes = await authApi.verifyEmail({
        email: testUser.email,
        code: dbCode,
      });
    }
  }

  if (!verifyRes.ok()) {
    const errorText = await verifyRes.text();
    throw new Error(`[AuthFixture] Email verification failed for ${testUser.email} with status ${verifyRes.status()}: ${errorText}`);
  }

  // 3. Real API Login
  const loginRes = await authApi.login({
    email: testUser.email,
    password: testUser.password,
  });

  if (!loginRes.ok()) {
    const errorText = await loginRes.text();
    throw new Error(`[AuthFixture] Login failed for ${testUser.email} with status ${loginRes.status()}: ${errorText}`);
  }

  const loginData = await loginRes.json();
  const token = loginData.access_token;
  if (!token) {
    throw new Error(`[AuthFixture] Login response did not contain access_token`);
  }

  // 4. Create Authenticated API Request Context
  const authContext = await playwright.request.newContext({
    baseURL: getNormalizedApiBaseUrl(),
    extraHTTPHeaders: {
      Authorization: `Bearer ${token}`,
    },
  });

  const session: AuthenticatedUserSession = {

```

```
    user: {
      id: userId,
      email: testUser.email,
      name: testUser.fullName,
      password: testUser.password,
    },
    token,
    authContext,
    api: {
      auth: new AuthApi(authContext),
      users: new UsersApi(authContext),
      wallet: new WalletApi(authContext),
      cards: new CardsApi(authContext),
      beneficiaries: new BeneficiariesApi(authContext),
      transfers: new TransfersApi(authContext),
      transactions: new TransactionsApi(authContext),
      exchangeRates: new ExchangeRatesApi(authContext),
    },
  },
};

await use(session);

// Teardown
await authContext.dispose();
},

db: async ({}, use) => {
  await use(dbClient);
},

redis: async ({}, use) => {
  await use(redisClient);
},

queue: async ({}, use) => {
  await use(transferQueueClient);
},
});

export { expect } from '@playwright/test';
```

## Code Explanation: Complete Provisioning Lifecycle

1. **Signup:** Executes real `POST /auth/signup` against NestJS backend using data from `generateTestUserData()`.
2. **Verification:** Uses development OTP `'123456'`. If running against staging or production where OTP is dynamically generated, it falls back to querying `email_verifications` table in PostgreSQL.
3. **Login:** Executes `POST /auth/login` to obtain the cryptographic JWT `access_token`.
4. **Context Creation:** Calls `playwright.request.newContext({...})` binding `Authorization: Bearer ${token}` as a default header on every outbound request.
5. **Client Instantiation:** Wraps `authContext` in domain API client instances ( `authUser.api.cards` , `authUser.api.wallet` , etc.).
6. **Teardown:** When the test finishes, `await authContext.dispose()` releases the HTTP connection pool and sockets.

## Part 9 — Infrastructure Smoke Tests

### File: qa/automation/tests/infra-smoke.spec.ts

#### Purpose

Smoke tests verifying that the underlying data stores (PostgreSQL, Redis) and messaging queues (BullMQ) are reachable and healthy before running functional suites.

#### Actual Current Code

```
import { test, expect } from '@playwright/test';
import { dbClient } from '../database/db-client';
import { redisClient } from '../redis/redis-client';
import { transferQueueClient, TRANSFERS_QUEUE_NAME } from '../queues/queue-client';

test.describe('Infrastructure Helpers Smoke Test', () => {

  test('PostgreSQL connection works via DbClient', async () => {
    const isHealthy = await dbClient.healthCheck();
    expect(isHealthy).toBe(true);

    const result = await dbClient.queryOne<{ alive: number }>('SELECT 1 as alive');
    expect(result).not.toBeNull();
    expect(result?.alive).toBe(1);
  });

  test('Redis connection works via RedisClient', async () => {
    const isHealthy = await redisClient.healthCheck();
    expect(isHealthy).toBe(true);

    // Verify safe isolated key operation with cleanup
    const testKey = `wrightpay:qa:smoke:${Date.now()}`;
    await redisClient.set(testKey, 'ok', 10);
    const exists = await redisClient.exists(testKey);
    expect(exists).toBe(true);

    const val = await redisClient.get(testKey);
    expect(val).toBe('ok');

    await redisClient.del(testKey);
    const existsAfterDel = await redisClient.exists(testKey);
    expect(existsAfterDel).toBe(false);
  });

  test('BullMQ transfers queue can be inspected via TransferQueueClient', async () => {
    const isHealthy = await transferQueueClient.healthCheck();
    expect(isHealthy).toBe(true);

    const queue = transferQueueClient.getQueue();
    expect(queue.name).toBe(TRANSFERS_QUEUE_NAME);

    const counts = await transferQueueClient.getJobCounts();
    expect(counts).toBeDefined();
    expect(typeof counts.active).toBe('number');
    expect(typeof counts.completed).toBe('number');
    expect(typeof counts.failed).toBe('number');
  });
});
```

#### Diagnostic Value of Smoke Test Failures

- **PostgreSQL Failure:** If Test 1 fails, PostgreSQL container on port 5432 is stopped, bad credentials in `DATABASE_URL`, or max connections exceeded.

- **Redis Failure:** If Test 2 fails, Redis container on port 6379 is down or network unreachable.
- **BullMQ Failure:** If Test 3 fails, Redis is running but BullMQ cannot read or write queue keys, or queue name is mismatched.

## Part 10 — API Smoke Tests

### File: qa/automation/tests/api-smoke.spec.ts

#### Purpose

Validates that the Playwright HTTP pipeline, unauthenticated API routing, and the `authUser` fixture are fully operational.

#### Actual Current Code

```
import { test, expect } from '../fixtures/api.fixtures';

test.describe('API Architecture Smoke Validation', () => {
  test('unauthenticated client can access public exchange-rates endpoint', async ({ exchangeRatesApi }) => {
    const response = await exchangeRatesApi.getAllRates();
    expect(response.status()).toBe(200);

    const body = await response.json();
    expect(Array.isArray(body)).toBe(true);
  });

  test('authUser fixture provisions real user and authenticated clients work', async ({ authUser }) => {
    expect(authUser.token).toBeDefined();
    expect(authUser.user.id).toBeDefined();
    expect(authUser.user.email).toContain('@wrightpay-qa.test');

    // Verify authenticated user profile endpoint
    const profileRes = await authUser.api.users.getMe();
    expect(profileRes.status()).toBe(200);
    const profile = await profileRes.json();
    expect(profile.id).toBe(authUser.user.id);
    expect(profile.email).toBe(authUser.user.email);

    // Verify authenticated wallet endpoint
    const walletRes = await authUser.api.wallet.getMyWallet();
    expect(walletRes.status()).toBe(200);
    const wallet = await walletRes.json();
    expect(wallet.id).toBeDefined();
    expect(wallet.currency).toBe('EUR');
    expect(wallet.balance).toBe(0);
    expect(wallet.equivalents).toBeDefined();
  });
});
```

### File: qa/automation/tests/proof-of-life.spec.ts

#### Purpose

Basic baseline assertion to confirm Playwright test discovery and runner pipeline execution.

## Actual Current Code

```
import { test, expect } from '@playwright/test';

test.describe('Automation Framework Skeleton Verification', () => {
  test('proof of life: test runner executes and assertion passes', async () => {
    // Basic assertion to confirm Playwright test discovery and execution pipeline
    const frameworkName = 'WrightPay Playwright Automation Framework';
    expect(frameworkName).toBeDefined();
    expect(frameworkName).toContain('WrightPay');
    expect(1 + 1).toBe(2);
  });
});
```

---

## Part 11 — Authentication Test Suite

---

### File: qa/automation/tests/auth/auth.spec.ts

---

#### Purpose

Validates all signup, login, email verification, logout, password hashing, and token boundary security invariants.

## Actual Current Code

```
import { test, expect } from '../../../fixtures/api.fixtures';
import { generateTestUserData } from '../../../test-data/user.factory';
import { dbClient } from '../../../database/db-client';

test.describe('Authentication Domain API Tests', () => {
  // =====
  // 1. POST /auth/signup
  // =====
  test.describe('POST /auth/signup', () => {
    test('successfully registers a new user with valid data and initializes database records', async ({ authApi }) =
> {
      const testUser = generateTestUserData();

      const response = await authApi.signup(testUser);
      expect(response.status()).toBe(201);

      const body = await response.json();
      expect(body).toBeDefined();
      expect(body.message).toBe('Signup successful. Please verify your email.');
```

expect(typeof body.userId).toBe('string');

expect(body.userId.length).toBeGreaterThan(0);

// Cross-layer PostgreSQL validation

```
const userRow = await dbClient.queryOne<{
  id: string;
  name: string;
  email: string;
  accountStatus: string;
  defaultCurrency: string;
  passwordHash: string;
}>(
  'SELECT id, name, email, "accountStatus", "defaultCurrency", "passwordHash" FROM users WHERE id = $1',
  [body.userId],
);

expect(userRow).not.toBeNull();
expect(userRow?.name).toBe(testUser.fullName);
expect(userRow?.email).toBe(testUser.email.toLowerCase());
expect(userRow?.accountStatus).toBe('pending');
expect(userRow?.defaultCurrency).toBe('EUR');
```

// Security check: password must be hashed (never stored plaintext)

```
expect(userRow?.passwordHash).not.toBe(testUser.password);
expect(userRow?.passwordHash.startsWith('$argon2')).toBe(true);
```

// Verify default EUR wallet is initialized with zero balance

```
const walletRow = await dbClient.queryOne<{
  id: string;
  userId: string;
  currency: string;
  balance: string | number;
  isDefault: boolean;
}>(
  'SELECT id, "userId", currency, balance, "isDefault" FROM wallets WHERE "userId" = $1',
  [body.userId],
);

expect(walletRow).not.toBeNull();
expect(walletRow?.userId).toBe(body.userId);
expect(walletRow?.currency).toBe('EUR');
expect(Number(walletRow?.balance)).toBe(0);
expect(walletRow?.isDefault).toBe(true);
});

test('rejects duplicate user registration with identical email', async ({ authApi }) => {
  const testUser = generateTestUserData();

  // First registration
  const firstRes = await authApi.signup(testUser);
  expect(firstRes.status()).toBe(201);
```

```

    // Duplicate attempt
    const duplicateRes = await authApi.signup(testUser);
    expect(duplicateRes.status()).toBe(400);

    const errorBody = await duplicateRes.json();
    expect(errorBody.message).toContain('Email already in use');
  });

  test('rejects registration when required fields are missing', async ({ authApi }) => {
    const baseUser = generateTestUserData();

    // Missing password
    const { password, ...withoutPassword } = baseUser;
    const resWithoutPassword = await authApi.signup(withoutPassword as any);
    expect(resWithoutPassword.status()).toBe(400);

    // Missing firstName
    const { firstName, ...withoutFirstName } = baseUser;
    const resWithoutFirstName = await authApi.signup(withoutFirstName as any);
    expect(resWithoutFirstName.status()).toBe(400);
  });

  test('rejects registration with malformed email format', async ({ authApi }) => {
    const invalidUser = generateTestUserData({ email: 'not-an-email-format' });
    const response = await authApi.signup(invalidUser);
    expect(response.status()).toBe(400);
  });

  test('rejects registration with password shorter than 8 characters', async ({ authApi }) => {
    const invalidUser = generateTestUserData({ password: 'short' });
    const response = await authApi.signup(invalidUser);
    expect(response.status()).toBe(400);
  });
});

// =====
// 2. POST /auth/login
// =====
test.describe('POST /auth/login', () => {
  test('successfully authenticates with valid credentials and returns JWT', async ({ authApi }) => {
    const testUser = generateTestUserData();
    const signupRes = await authApi.signup(testUser);
    expect(signupRes.status()).toBe(201);

    const loginRes = await authApi.login({
      email: testUser.email,
      password: testUser.password,
    });

    expect(loginRes.status()).toBe(200);
    const body = await loginRes.json();
    expect(body).toBeDefined();

    // Token assertion (never print or hardcode token value)
    expect(typeof body.access_token).toBe('string');
    expect(body.access_token.length).toBeGreaterThan(20);

    // User object assertion
    expect(body.user).toBeDefined();
    expect(typeof body.user.id).toBe('string');
    expect(body.user.email).toBe(testUser.email.toLowerCase());
    expect(body.user.name).toBe(testUser.fullName);
  });

  test('rejects login with incorrect password', async ({ authApi }) => {
    const testUser = generateTestUserData();
    await authApi.signup(testUser);

    const loginRes = await authApi.login({
      email: testUser.email,
      password: 'IncorrectPassword999!',
    });

    expect(loginRes.status()).toBe(401);
  });
});

```

```

    const errorBody = await loginRes.json();
    expect(errorBody.message).toBe('INVALID_CREDENTIALS');
  });

  test('rejects login with non-existent email', async ({ authApi }) => {
    const nonExistentEmail = `unregistered_${Date.now()}@wrightpay-qa.test`;
    const loginRes = await authApi.login({
      email: nonExistentEmail,
      password: 'SomePassword123!',
    });

    expect(loginRes.status()).toBe(401);
    const errorBody = await loginRes.json();
    expect(errorBody.message).toBe('INVALID_CREDENTIALS');
  });

  test('rejects login with missing credentials', async ({ authApi }) => {
    const res = await authApi.login({ email: '', password: '' });
    expect(res.status()).toBe(400);
  });
});

// =====
// 3. POST /auth/verify-email
// =====
test.describe('POST /auth/verify-email', () => {
  test('successfully verifies email and transitions accountStatus to active in database', async ({ authApi }) => {
    const testUser = generateTestUserData();
    const signupRes = await authApi.signup(testUser);
    const { userId } = await signupRes.json();

    // Verify user starts in pending status
    const beforeRow = await dbClient.queryOne<{ accountStatus: string }>(
      'SELECT "accountStatus" FROM users WHERE id = $1',
      [userId],
    );
    expect(beforeRow?.accountStatus).toBe('pending');

    // Development OTP is 123456
    const verifyRes = await authApi.verifyEmail({
      email: testUser.email,
      code: '123456',
    });

    expect(verifyRes.status()).toBe(200);
    const body = await verifyRes.json();
    expect(body.message).toBe('Email successfully verified');

    // Verify accountStatus transitioned to active
    const afterRow = await dbClient.queryOne<{ accountStatus: string }>(
      'SELECT "accountStatus" FROM users WHERE id = $1',
      [userId],
    );
    expect(afterRow?.accountStatus).toBe('active');
  });

  test('rejects email verification with incorrect OTP code', async ({ authApi }) => {
    const testUser = generateTestUserData();
    await authApi.signup(testUser);

    const verifyRes = await authApi.verifyEmail({
      email: testUser.email,
      code: '999999',
    });

    expect(verifyRes.status()).toBe(400);
    const errorBody = await verifyRes.json();
    expect(errorBody.message).toContain('INVALID_INPUT');
  });

  test('rejects re-verification of an already verified email', async ({ authApi }) => {
    const testUser = generateTestUserData();
    await authApi.signup(testUser);
  });
});

```

```

    // First verification succeeds
    const firstRes = await authApi.verifyEmail({
      email: testUser.email,
      code: '123456',
    });
    expect(firstRes.status()).toBe(200);

    // Second verification attempt fails
    const secondRes = await authApi.verifyEmail({
      email: testUser.email,
      code: '123456',
    });
    expect(secondRes.status()).toBe(400);
    const errorBody = await secondRes.json();
    expect(errorBody.message).toContain('Email is already verified');
  });

  test('rejects email verification with invalid code format', async ({ authApi }) => {
    const testUser = generateTestUserData();
    await authApi.signup(testUser);

    // Code must be exactly 6 characters per DTO length validation
    const verifyRes = await authApi.verifyEmail({
      email: testUser.email,
      code: '123',
    });

    expect(verifyRes.status()).toBe(400);
  });
});

// =====
// 4. POST /auth/logout
// =====
test.describe('POST /auth/logout', () => {
  test('successfully acknowledges logout for authenticated user', async ({ authUser }) => {
    const logoutRes = await authUser.api.auth.logout();
    expect(logoutRes.status()).toBe(200);

    const body = await logoutRes.json();
    expect(body.message).toBe('Logged out successfully');
  });

  test('rejects logout request when Authorization header is omitted', async ({ authApi }) => {
    const logoutRes = await authApi.logout();
    expect(logoutRes.status()).toBe(401);
  });
});

// =====
// 5. Authentication Security & Boundaries
// =====
test.describe('Authentication Security Boundaries', () => {
  test('protected endpoint returns 401 when Authorization header is missing', async ({ apiContext }) => {
    const response = await apiContext.get('users/me');
    expect(response.status()).toBe(401);
  });

  test('protected endpoint returns 401 when scheme is not Bearer', async ({ apiContext }) => {
    const response = await apiContext.get('users/me', {
      headers: {
        Authorization: 'Basic dXNlcjpwYXNzd29yZA==',
      },
    });
    expect(response.status()).toBe(401);
  });

  test('protected endpoint returns 401 when Bearer token is malformed', async ({ apiContext }) => {
    const response = await apiContext.get('users/me', {
      headers: {
        Authorization: 'Bearer invalid.malformed.jwttoken',
      },
    });
    expect(response.status()).toBe(401);
  });
});

```

```
});

test('protected endpoint returns 401 when token is tampered with forged signature', async ({ apiContext }) => {
  // Valid structural base64 header & payload with bogus signature
  const header = Buffer.from(JSON.stringify({ alg: 'HS256', typ: 'JWT' })).toString('base64url');
  const payload = Buffer.from(JSON.stringify({ sub: '00000000-0000-0000-0000-000000000000', email: 'forged@test.com' })).toString('base64url');
  const forgedJwt = `${header}.${payload}.invalidSignatureHere`;

  const response = await apiContext.get('users/me', {
    headers: {
      Authorization: `Bearer ${forgedJwt}`,
    },
  });
  expect(response.status()).toBe(401);
});
});
});
```

## Detailed Breakdown of All 19 Test Cases

### Test 1: successfully registers a new user with valid data and initializes database records

- Behavior Tested:** Complete user registration happy path, return contract, Argon2 password hashing, and default wallet initialization.
- Why It Exists:** Verifies the root onboarding contract of WrightPay.
- Request Sent:** `POST /auth/signup`
- Data Sent:** Unique `TestUserData` with `firstName`, `lastName`, `email`, `password`, `agreeTerms: true`.
- Expected Response:** `201 Created` with JSON `{ message: "Signup successful. Please verify your email.", userId: "<uuid>" }`.
- Assertions Performed:** Status 201, message correctness, non-empty string `userId`.
- Database Verification Performed:** Yes. Queries `users` and `wallets` tables.
- Why DB Verification Matters:** Validates that `accountStatus` is initialized to `'pending'`, `defaultCurrency` is `'EUR'`, `passwordHash` starts with `$argon2` (never plaintext), and a default `wallets` row is created with zero balance.
- Rule Tested:** Onboarding integrity, data protection compliance (GDPR/PCI-DSS), and relational integrity.
- Catchable Bug:** Passwords leaked into plaintext columns or failed default wallet creation.

### Test 2: rejects duplicate user registration with identical email

- Behavior Tested:** Duplicate email uniqueness constraint handling.
- Why It Exists:** Prevents account takeover and duplicate identity records.
- Request Sent:** Two consecutive `POST /auth/signup` requests with identical emails.
- Data Sent:** Identical `testUser` object.
- Expected Response:** First request returns `201 Created`; second returns `400 Bad Request`.
- Assertions Performed:** Status 400, message contains `"Email already in use"`.
- Database Verification:** None required (asserted via HTTP contract).
- Why DB Verification Matters:** Uniqueness constraint in PostgreSQL is reflected via HTTP 400 rather than an unhandled 500 server crash.
- Rule Tested:** One account per verified email.
- Catchable Bug:** Database constraint violation causing uncaught 500 internal server errors.

### Test 3: rejects registration when required fields are missing

- Behavior Tested:** Request validation boundaries when omitting `password` or `firstName`.
- Why It Exists:** Verifies NestJS `ValidationPipe` enforcement.
- Request Sent:** `POST /auth/signup`
- Data Sent:** Payloads missing `password` and `firstName` respectively.

5. **Expected Response:** 400 Bad Request
6. **Assertions Performed:** `expect(res.status()).toBe(400)`
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Invalid payloads must be stopped before hitting PostgreSQL.
9. **Rule Tested:** Mandatory field presence.
10. **Catchable Bug:** Null values causing database crashes or silent user creation with null passwords.

**Test 4:** rejects registration with malformed email format

1. **Behavior Tested:** Email string format validation ( `@IsEmail()` ).
2. **Why It Exists:** Prevents garbage email strings in database.
3. **Request Sent:** POST /auth/signup
4. **Data Sent:** email: 'not-an-email-format'
5. **Expected Response:** 400 Bad Request
6. **Assertions Performed:** Status 400.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Stops non-RFC compliant emails.
9. **Rule Tested:** Strict email format validation.
10. **Catchable Bug:** Corrupted contact data breaking downstream mailers.

**Test 5:** rejects registration with password shorter than 8 characters

1. **Behavior Tested:** Minimum password length enforcement ( `@MinLength(8)` ).
2. **Why It Exists:** Enforces baseline password complexity.
3. **Request Sent:** POST /auth/signup
4. **Data Sent:** password: 'short' (5 characters)
5. **Expected Response:** 400 Bad Request
6. **Assertions Performed:** Status 400.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Security policy must be enforced at the gateway.
9. **Rule Tested:** Passwords must be at least 8 characters.
10. **Catchable Bug:** Weak passwords allowed.

**Test 6:** successfully authenticates with valid credentials and returns JWT

1. **Behavior Tested:** Login happy path and JWT issuance.
2. **Why It Exists:** Core authentication mechanism for all downstream protected endpoints.
3. **Request Sent:** POST /auth/login
4. **Data Sent:** { email, password } matching previously signed-up user.
5. **Expected Response:** 200 OK with { access\_token, user: { id, email, name } }.
6. **Assertions Performed:** Status 200, access\_token string length > 20, user ID and name matching.
7. **Database Verification:** None (indirectly verified via previous signup).
8. **Why DB Verification Matters:** Proves credential lookup against PostgreSQL hashes works.
9. **Rule Tested:** Correct password verifies against Argon2 hash.
10. **Catchable Bug:** Broken JWT signing or payload corruption.

**Test 7:** rejects login with incorrect password

1. **Behavior Tested:** Credential validation failure.
2. **Why It Exists:** Blocks brute-force and unauthorized access.
3. **Request Sent:** POST /auth/login

4. **Data Sent:** Registered email with incorrect password `'IncorrectPassword999!'`.
5. **Expected Response:** `401 Unauthorized` with `{ message: "INVALID_CREDENTIALS" }`.
6. **Assertions Performed:** Status 401, error message check.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Prevents credential compromise.
9. **Rule Tested:** Reject invalid passwords with generic error.
10. **Catchable Bug:** Accepting invalid passwords or returning 200 with blank tokens.

**Test 8:** `rejects login with non-existent email`

1. **Behavior Tested:** Login rejection on unknown account.
2. **Why It Exists:** Prevents unauthorized login on non-existent identities.
3. **Request Sent:** `POST /auth/login`
4. **Data Sent:** Unregistered random email address.
5. **Expected Response:** `401 Unauthorized` with `{ message: "INVALID_CREDENTIALS" }`.
6. **Assertions Performed:** Status 401, error message is identical to wrong-password error.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Timing/user enumeration protection (both wrong email and wrong password return `INVALID_CREDENTIALS`).
9. **Rule Tested:** Safe error disclosure without leaking account existence.
10. **Catchable Bug:** Account enumeration vulnerability where unknown email returns 404 instead of 401.

**Test 9:** `rejects login with missing credentials`

1. **Behavior Tested:** DTO validation when email or password is empty.
2. **Why It Exists:** Prevents empty payload execution.
3. **Request Sent:** `POST /auth/login`
4. **Data Sent:** `{ email: '', password: '' }`
5. **Expected Response:** `400 Bad Request`
6. **Assertions Performed:** Status 400.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Rejects request prior to database lookup.
9. **Rule Tested:** Mandatory input fields on login.
10. **Catchable Bug:** Empty credentials triggering unexpected database exceptions.

**Test 10:** `successfully verifies email and transitions accountStatus to active in database`

1. **Behavior Tested:** OTP email confirmation and account activation state machine.
2. **Why It Exists:** WrightPay requires verified accounts before conducting financial transfers.
3. **Request Sent:** `POST /auth/verify-email`
4. **Data Sent:** `{ email, code: '123456' }`
5. **Expected Response:** `200 OK` with `{ message: "Email successfully verified" }`.
6. **Assertions Performed:** Status 200, confirmation message.
7. **Database Verification Performed:** Yes. Queries `users.accountStatus` before and after.
8. **Why DB Verification Matters:** Confirms status transitions from `'pending'` to `'active'` in PostgreSQL.
9. **Rule Tested:** Email verification unlocks full account status.
10. **Catchable Bug:** User status remains `'pending'` despite successful OTP submission.

**Test 11:** `rejects email verification with incorrect OTP code`

1. **Behavior Tested:** OTP verification rejection on bad code.

2. **Why It Exists:** Blocks brute-force OTP attempts.
3. **Request Sent:** `POST /auth/verify-email`
4. **Data Sent:** `{ email, code: '999999' }`
5. **Expected Response:** `400 Bad Request` with `INVALID_INPUT`.
6. **Assertions Performed:** Status 400, message contains `INVALID_INPUT`.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** OTP must match persisted/expected secret.
9. **Rule Tested:** Rejection of invalid verification tokens.
10. **Catchable Bug:** Bypassed OTP verification.

**Test 12:** `rejects re-verification of an already verified email`

1. **Behavior Tested:** Idempotency and replay protection on email verification.
2. **Why It Exists:** Prevents reusing OTPs or re-verifying active accounts.
3. **Request Sent:** Two consecutive `POST /auth/verify-email` calls.
4. **Data Sent:** `{ email, code: '123456' }`
5. **Expected Response:** First returns 200; second returns `400 Bad Request` with `"Email is already verified"`.
6. **Assertions Performed:** Status 400, message assertion.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** State machine rule prevents re-triggering activation logic.
9. **Rule Tested:** Verification is single-use.
10. **Catchable Bug:** OTP replay attack.

**Test 13:** `rejects email verification with invalid code format`

1. **Behavior Tested:** Length validation on OTP code ( `@Length(6, 6)` ).
2. **Why It Exists:** Prevents truncated or malformed OTP codes.
3. **Request Sent:** `POST /auth/verify-email`
4. **Data Sent:** `{ email, code: '123' }` (3 digits instead of 6)
5. **Expected Response:** `400 Bad Request`
6. **Assertions Performed:** Status 400.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** DTO validation catches format errors early.
9. **Rule Tested:** OTP codes must be exactly 6 characters.
10. **Catchable Bug:** SQL injection or buffer overflow via oversized/undersized codes.

**Test 14:** `successfully acknowledges logout for authenticated user`

1. **Behavior Tested:** Authenticated session termination.
2. **Why It Exists:** Verifies logout endpoint for authenticated users.
3. **Request Sent:** `POST /auth/logout` with valid Bearer token.
4. **Data Sent:** None.
5. **Expected Response:** `200 OK` with `{ message: "Logged out successfully" }`.
6. **Assertions Performed:** Status 200, message check.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Confirms endpoint contract.
9. **Rule Tested:** Authenticated logout contract.
10. **Catchable Bug:** Logout crashing or requiring unexpected payload.

**Test 15: rejects logout request when Authorization header is omitted**

1. **Behavior Tested:** Protected route guard on logout.
2. **Why It Exists:** Logout requires an active session to terminate.
3. **Request Sent:** `POST /auth/logout` without `Authorization` header.
4. **Data Sent:** None.
5. **Expected Response:** `401 Unauthorized`
6. **Assertions Performed:** Status 401.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Guards route against anonymous invocation.
9. **Rule Tested:** `JwtAuthGuard` protection.
10. **Catchable Bug:** Unprotected logout route.

**Test 16: protected endpoint returns 401 when Authorization header is missing**

1. **Behavior Tested:** Baseline route protection against unauthenticated traffic.
2. **Why It Exists:** Ensures all sensitive endpoints require identity tokens.
3. **Request Sent:** `GET /users/me` without `Authorization` header.
4. **Data Sent:** None.
5. **Expected Response:** `401 Unauthorized`
6. **Assertions Performed:** Status 401.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Universal security baseline.
9. **Rule Tested:** Missing token yields 401.
10. **Catchable Bug:** Accidental omission of `@UseGuards(JwtAuthGuard)` on controller routes.

**Test 17: protected endpoint returns 401 when scheme is not Bearer**

1. **Behavior Tested:** Authentication scheme enforcement.
2. **Why It Exists:** Ensures API accepts only Bearer tokens and rejects other schemes (like Basic).
3. **Request Sent:** `GET /users/me` with `Authorization: Basic dXNlcjpwYXNzd29yZA==`.
4. **Data Sent:** None.
5. **Expected Response:** `401 Unauthorized`
6. **Assertions Performed:** Status 401.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Validates authentication protocol compliance.
9. **Rule Tested:** Strict Bearer token authentication scheme.
10. **Catchable Bug:** Backend misinterpreting non-Bearer schemes.

**Test 18: protected endpoint returns 401 when Bearer token is malformed**

1. **Behavior Tested:** Token parser resilience against malformed strings.
2. **Why It Exists:** Prevents crashes when invalid strings are passed in header.
3. **Request Sent:** `GET /users/me` with `Authorization: Bearer invalid.malformed.jwttoken`.
4. **Data Sent:** None.
5. **Expected Response:** `401 Unauthorized`
6. **Assertions Performed:** Status 401.
7. **Database Verification:** None.
8. **Why DB Verification Matters:** Ensures JWT parser fails gracefully without crashing the server.
9. **Rule Tested:** Invalid JWT structure must result in 401.
10. **Catchable Bug:** Unhandled token parse exceptions throwing 500 error.

**Test 19:** `protected endpoint returns 401 when token is tampered with forged signature`

1. **Behavior Tested:** Cryptographic signature verification.
  2. **Why It Exists:** Prevents attackers from forging identity claims in the JWT payload.
  3. **Request Sent:** `GET /users/me` with structurally valid Base64 header and payload ( `sub: 00000000-...`, `email: forged@test.com` ), but with forged signature `'invalidSignatureHere'`.
  4. **Data Sent:** None.
  5. **Expected Response:** `401 Unauthorized`
  6. **Assertions Performed:** Status 401.
  7. **Database Verification:** None.
  8. **Why DB Verification Matters:** Confirms backend verifies HMAC SHA-256 signature against its server secret.
  9. **Rule Tested:** Signature tampering must be cryptographically rejected.
  10. **Catchable Bug:** Token signature verification disabled ( `algorithms: ['none']` or missing secret).
- 

## Part 12 — Users Test Suite

---

### File: `qa/automation/tests/users/users.spec.ts`

---

#### Purpose

Tests user profile retrieval ( `GET /users/me` ), profile updates ( `PATCH /users/me` ), parameter tampering protection, and cross-user isolation.

## Actual Current Code

```
import { test, expect } from '../../../fixtures/api.fixtures';
import { generateTestUserData } from '../../../test-data/user.factory';
import { dbClient } from '../../../database/db-client';
import { UsersApi } from '../../../api';

test.describe('Users Domain API Tests', () => {
  // =====
  // 1. GET /users/me
  // =====
  test.describe('GET /users/me', () => {
    test('successfully retrieves authenticated user profile with correct identity fields', async ({ authUser }) => {
      const response = await authUser.api.users.getMe();
      expect(response.status()).toBe(200);

      const body = await response.json();
      expect(body).toBeDefined();

      // Identity fields verification
      expect(body.id).toBe(authUser.user.id);
      expect(body.email).toBe(authUser.user.email);
      expect(body.name).toBe(authUser.user.name);
      expect(body.accountType).toBe('individual');
      expect(body.accountStatus).toBe('active');
      expect(body.defaultCurrency).toBe('EUR');
      expect(body.createdAt).toBeDefined();
      expect(typeof body.createdAt).toBe('string');

      // Security check: sensitive fields must not be exposed in the profile response
      expect(body.passwordHash).toBeUndefined();
      expect(body.password).toBeUndefined();
    });

    test('maintains strict user isolation across separate authenticated sessions', async ({
      authUser,
      authApi,
      playwright,
    }) => {
      // User A from fixture
      const responseA = await authUser.api.users.getMe();
      expect(responseA.status()).toBe(200);
      const profileA = await responseA.json();

      // Provision User B independently
      const testUserB = generateTestUserData();
      const signupB = await authApi.signup(testUserB);
      expect(signupB.status()).toBe(201);
      const { userId: userIdB } = await signupB.json();

      await authApi.verifyEmail({ email: testUserB.email, code: '123456' });

      const loginB = await authApi.login({
        email: testUserB.email,
        password: testUserB.password,
      });
      expect(loginB.status()).toBe(200);
      const { access_token: tokenB } = await loginB.json();

      // Create isolated request context for User B
      const contextB = await playwright.request.newContext({
        baseURL: responseA.url().replace(/\/users\/me.*$/, '/'),
        extraHTTPHeaders: {
          Authorization: `Bearer ${tokenB}`,
        },
      });

      const usersApiB = new UsersApi(contextB);
      const responseB = await usersApiB.getMe();
      expect(responseB.status()).toBe(200);
      const profileB = await responseB.json();

      // Verify User A receives User A's identity and User B receives User B's identity
    });
  });
});
```

```

    expect(profileA.id).toBe(authUser.user.id);
    expect(profileA.email).toBe(authUser.user.email);

    expect(profileB.id).toBe(userIdB);
    expect(profileB.email).toBe(testUserB.email.toLowerCase());

    // Assert complete identity isolation between users
    expect(profileA.id).not.toBe(profileB.id);
    expect(profileA.email).not.toBe(profileB.email);

    await contextB.dispose();
  });

  test('rejects unauthenticated request when Authorization header is omitted', async ({ apiContext }) => {
    const response = await apiContext.get('users/me');
    expect(response.status()).toBe(401);
  });

  test('rejects request with invalid Bearer token', async ({ apiContext }) => {
    const response = await apiContext.get('users/me', {
      headers: {
        Authorization: 'Bearer invalid.token.string',
      },
    });
    expect(response.status()).toBe(401);
  });
});

// =====
// 2. PATCH /users/me
// =====
test.describe('PATCH /users/me', () => {
  test('successfully updates user profile name and countryOfResidence', async ({ authUser }) => {
    const updatedName = 'Alex Mercer';
    const updatedCountry = 'Germany';

    const patchResponse = await authUser.api.users.updateMe({
      name: updatedName,
      countryOfResidence: updatedCountry,
    });

    expect(patchResponse.status()).toBe(200);
    const updatedProfile = await patchResponse.json();
    expect(updatedProfile.name).toBe(updatedName);
    expect(updatedProfile.countryOfResidence).toBe(updatedCountry);
    expect(updatedProfile.id).toBe(authUser.user.id);

    // Verify update is observable via subsequent GET /users/me
    const getResponse = await authUser.api.users.getMe();
    expect(getResponse.status()).toBe(200);
    const fetchedProfile = await getResponse.json();
    expect(fetchedProfile.name).toBe(updatedName);
    expect(fetchedProfile.countryOfResidence).toBe(updatedCountry);

    // Cross-layer PostgreSQL verification
    const dbRow = await dbClient.queryOne<{ name: string; countryOfResidence: string }>(
      'SELECT name, "countryOfResidence" FROM users WHERE id = $1',
      [authUser.user.id],
    );
    expect(dbRow).not.toBeNull();
    expect(dbRow?.name).toBe(updatedName);
    expect(dbRow?.countryOfResidence).toBe(updatedCountry);
  });

  test('successfully updates user default currency', async ({ authUser }) => {
    const patchResponse = await authUser.api.users.updateMe({
      defaultCurrency: 'GBP',
    });

    expect(patchResponse.status()).toBe(200);
    const updatedProfile = await patchResponse.json();
    expect(updatedProfile.defaultCurrency).toBe('GBP');

    // Cross-layer PostgreSQL verification

```

```

    const dbRow = await dbClient.queryOne<{ defaultCurrency: string }>(
      'SELECT "defaultCurrency" FROM users WHERE id = $1',
      [authUser.user.id],
    );
    expect(dbRow?.defaultCurrency).toBe('GBP');
  });

  test('accepts empty update payload and returns unchanged profile', async ({ authUser }) => {
    const beforeRes = await authUser.api.users.getMe();
    const beforeProfile = await beforeRes.json();

    const patchResponse = await authUser.api.users.updateMe({});
    expect(patchResponse.status()).toBe(200);
    const afterProfile = await patchResponse.json();

    expect(afterProfile.id).toBe(beforeProfile.id);
    expect(afterProfile.email).toBe(beforeProfile.email);
    expect(afterProfile.name).toBe(beforeProfile.name);
    expect(afterProfile.defaultCurrency).toBe(beforeProfile.defaultCurrency);
  });

  test('rejects update with invalid currency enum value', async ({ authUser }) => {
    const patchResponse = await authUser.api.users.updateMe({
      defaultCurrency: 'INVALID_CURRENCY' as any,
    });

    expect(patchResponse.status()).toBe(400);
  });

  test('rejects update with non-string name value', async ({ authUser }) => {
    const patchResponse = await authUser.api.users.updateMe({
      name: 12345 as any,
    });

    expect(patchResponse.status()).toBe(400);
  });

  test('prevents parameter tampering and IDOR attempts by stripping non-whitelisted fields', async ({ authUser })
=> {
    // Attempt to tamper with id and email through PATCH body
    const tamperAttempt = {
      id: '00000000-0000-0000-0000-000000000000',
      email: 'tampered_hacker@wrightpay-qa.test',
      name: 'Legit Name Update',
    };

    const patchResponse = await authUser.api.users.updateMe(tamperAttempt as any);
    expect(patchResponse.status()).toBe(200);

    const body = await patchResponse.json();
    // Whitelisted name is updated
    expect(body.name).toBe('Legit Name Update');
    // Immutable identity fields remain completely untouched
    expect(body.id).toBe(authUser.user.id);
    expect(body.id).not.toBe(tamperAttempt.id);
    expect(body.email).toBe(authUser.user.email);
    expect(body.email).not.toBe(tamperAttempt.email);

    // Verify in PostgreSQL that id and email were not tampered with
    const dbRow = await dbClient.queryOne<{ id: string; email: string }>(
      'SELECT id, email FROM users WHERE id = $1',
      [authUser.user.id],
    );
    expect(dbRow?.id).toBe(authUser.user.id);
    expect(dbRow?.email).toBe(authUser.user.email);
  });

  test('rejects unauthorized PATCH when Authorization header is omitted', async ({ apiContext }) => {
    const response = await apiContext.patch('users/me', {
      data: { name: 'Unauthorized Change' },
    });
    expect(response.status()).toBe(401);
  });

```

```
test('rejects PATCH with invalid Bearer token', async ({ apiContext }) => {
  const response = await apiContext.patch('users/me', {
    data: { name: 'Unauthorized Change' },
    headers: {
      Authorization: 'Bearer bogus.token.string',
    },
  });
  expect(response.status()).toBe(401);
});
});
});
```

## Detailed Breakdown of All 12 Test Cases

- **Test 1 ( GET /users/me Profile Retrieval & Sanitization )**: Asserts status 200, correct identity fields ( `id`, `email`, `name`, `accountType: 'individual'`, `accountStatus: 'active'`, `defaultCurrency: 'EUR'` ), and critically asserts that sensitive fields ( `password`, `passwordHash` ) are `undefined`.
- **Test 2 ( GET /users/me Multi-Tenant Isolation )**: Provisions User A and User B concurrently. Asserts that User A receives Profile A and User B receives Profile B with zero data leakage across sessions.
- **Test 3 & 4 (Negatives)**: Asserts 401 when Authorization is missing or invalid.
- **Test 5 ( PATCH /users/me Name & Country Update )**: Sends `{ name: 'Alex Mercer', countryOfResidence: 'Germany' }`. Asserts 200, subsequent `GET /users/me` parity, and cross-layer PostgreSQL row verification.
- **Test 6 ( PATCH /users/me Currency Update )**: Updates `defaultCurrency` to `'GBP'`. Asserts 200 and PostgreSQL database column update.
- **Test 7 ( PATCH /users/me Empty Payload )**: Sends `{}`. Asserts 200 and profile attributes remain completely unchanged.
- **Test 8 & 9 (Validation Boundaries)**: Sends invalid enum `'INVALID_CURRENCY'` and non-string `name: 12345`. Asserts 400 Bad Request.
- **Test 10 (Mass Assignment & Parameter Tampering)**: Injects `{ id: '00000000-...', email: 'tampered@hacker.test', name: 'Legit Name' }`. Verifies that NestJS `ValidationPipe` whitelist strips `id` and `email` while saving `name`. Verified via PostgreSQL query.
- **Test 11 & 12 (PATCH Negatives)**: Asserts 401 when attempting PATCH without token or with malformed token.

## Part 13 — Wallet Test Suite

### File: qa/automation/tests/wallet/wallet.spec.ts

#### Purpose

Tests wallet balance contract, numeric precision, database persistence parity, calculated multi-currency equivalents, and read idempotency.

## Actual Current Code

```
import { test, expect } from '../../../fixtures/api.fixtures';
import { generateTestUserData } from '../../../test-data/user.factory';
import { dbClient } from '../../../database/db-client';
import { WalletApi } from '../../../api';

test.describe('Wallet Domain API Tests', () => {
  // =====
  // 1. Authenticated Retrieval & Contract
  // =====
  test.describe('GET /wallets/me - Contract & Schema', () => {
    test('successfully retrieves authenticated wallet with valid schema and zero initial balance', async ({
      authUser,
    }) => {
      const response = await authUser.api.wallet.getMyWallet();
      expect(response.status()).toBe(200);

      const body = await response.json();
      expect(body).toBeDefined();

      // Structure and field assertions
      expect(typeof body.id).toBe('string');
      expect(body.id.length).toBeGreaterThan(0);
      expect(body.currency).toBe('EUR');
      expect(typeof body.balance).toBe('number');
      expect(body.balance).toBe(0);
      expect(body.isDefault).toBe(true);
      expect(typeof body.equivalents).toBe('object');
      expect(body.equivalents).not.toBeNull();

      // Sensitive / internal fields should not be exposed
      expect(body.userId).toBeUndefined();
    });

    test('balance representation maintains non-negative finite numeric precision', async ({ authUser }) => {
      const response = await authUser.api.wallet.getMyWallet();
      expect(response.status()).toBe(200);

      const body = await response.json();
      expect(Number.isFinite(body.balance)).toBe(true);
      expect(body.balance).toBeGreaterThanOrEqual(0);

      // Verify rounding precision (at most 2 decimal places)
      const decimalParts = body.balance.toString().split('.');
      if (decimalParts.length > 1) {
        expect(decimalParts[1].length).toBeLessThanOrEqual(2);
      }
    });
  });

  // =====
  // 2. Database Cross-Layer Validation
  // =====
  test.describe('GET /wallets/me - Database Cross-Layer Validation', () => {
    test('matches persisted PostgreSQL wallet record properties exactly', async ({ authUser }) => {
      const response = await authUser.api.wallet.getMyWallet();
      expect(response.status()).toBe(200);
      const apiWallet = await response.json();

      // Query database for the authenticated user's primary wallet
      const dbWallet = await dbClient.queryOne<{
        id: string;
        userId: string;
        currency: string;
        balance: string | number;
        isDefault: boolean;
      }>(
        'SELECT id, "userId", currency, balance, "isDefault" FROM wallets WHERE "userId" = $1',
        [authUser.user.id],
      );

      expect(dbWallet).not.toBeNull();
    });
  });
});
```

```

    expect(dbWallet?.id).toBe(apiWallet.id);
    expect(dbWallet?.userId).toBe(authUser.userId);
    expect(dbWallet?.currency).toBe(apiWallet.currency);
    expect(Number(dbWallet?.balance)).toBe(apiWallet.balance);
    expect(dbWallet?.isDefault).toBe(apiWallet.isDefault);
  });
});

// =====
// 3. Multi-Currency Equivalents Invariants
// =====
test.describe('GET /wallets/me - Currency Equivalents Invariants', () => {
  test('returns calculated equivalent balances for all supported currencies', async ({ authUser }) => {
    const response = await authUser.api.wallet.getMyWallet();
    expect(response.status()).toBe(200);
    const body = await response.json();

    // Expected supported currencies confirmed by WalletsService
    const expectedCurrencies = ['EUR', 'GBP', 'USD', 'AED', 'PLN', 'INR'];

    for (const curr of expectedCurrencies) {
      expect(body.equivalents).toHaveProperty(curr);
      expect(typeof body.equivalents[curr]).toBe('number');
      expect(body.equivalents[curr]).toBeGreaterThanOrEqual(0);
      expect(Number.isFinite(body.equivalents[curr])).toBe(true);
    }

    // Base currency equivalent must equal current wallet balance
    expect(body.equivalents[body.currency]).toBe(body.balance);
  });
});

// =====
// 4. User Isolation & Read-Only Idempotence
// =====
test.describe('GET /wallets/me - User Isolation & Idempotence', () => {
  test('ensures distinct users receive distinct wallets with strict separation', async ({
    authUser,
    authApi,
    playwright,
  }) => {
    // User A from fixture
    const responseA = await authUser.api.wallet.getMyWallet();
    expect(responseA.status()).toBe(200);
    const walletA = await responseA.json();

    // Provision User B
    const testUserB = generateTestUserData();
    const signupB = await authApi.signup(testUserB);
    expect(signupB.status()).toBe(201);
    const { userId: userIdB } = await signupB.json();

    await authApi.verifyEmail({ email: testUserB.email, code: '123456' });

    const loginB = await authApi.login({
      email: testUserB.email,
      password: testUserB.password,
    });
    expect(loginB.status()).toBe(200);
    const { access_token: tokenB } = await loginB.json();

    const contextB = await playwright.request.newContext({
      baseURL: responseA.url().replace(/\/wallets\/me.*$/, '/'),
      extraHTTPHeaders: {
        Authorization: `Bearer ${tokenB}`,
      },
    });

    const walletApiB = new WalletApi(contextB);
    const responseB = await walletApiB.getMyWallet();
    expect(responseB.status()).toBe(200);
    const walletB = await responseB.json();

    // Isolation assertions

```

```

    expect(walletA.id).not.toBe(walletB.id);

    // Verify in database that ownership maps to distinct users
    const dbWalletA = await dbClient.queryOne<{ userId: string }>('SELECT "userId" FROM wallets WHERE id = $1', [walletA.id]);
    const dbWalletB = await dbClient.queryOne<{ userId: string }>('SELECT "userId" FROM wallets WHERE id = $1', [walletB.id]);

    expect(dbWalletA?.userId).toBe(authUser.user.id);
    expect(dbWalletB?.userId).toBe(userIdB);
    expect(dbWalletA?.userId).not.toBe(dbWalletB?.userId);

    await contextB.dispose();
  });

  test('read operation is strictly idempotent and does not create duplicate wallet records', async ({
    authUser,
  }) => {
    // Check initial wallet count in DB for this user
    const beforeCount = await dbClient.queryOne<{ count: string }>(
      'SELECT count(*) FROM wallets WHERE "userId" = $1',
      [authUser.user.id],
    );
    expect(Number(beforeCount?.count)).toBe(1);

    // Perform multiple consecutive GET requests
    const res1 = await authUser.api.wallet.getMyWallet();
    expect(res1.status()).toBe(200);
    const wallet1 = await res1.json();

    const res2 = await authUser.api.wallet.getMyWallet();
    expect(res2.status()).toBe(200);
    const wallet2 = await res2.json();

    // Confirm properties remain identical
    expect(wallet1.id).toBe(wallet2.id);
    expect(wallet1.currency).toBe(wallet2.currency);
    expect(wallet1.balance).toBe(wallet2.balance);

    // Confirm no additional records were created in database
    const afterCount = await dbClient.queryOne<{ count: string }>(
      'SELECT count(*) FROM wallets WHERE "userId" = $1',
      [authUser.user.id],
    );
    expect(Number(afterCount?.count)).toBe(1);
  });
});

// =====
// 5. Authorization Negatives
// =====
test.describe('GET /wallets/me - Authorization Negatives', () => {
  test('rejects unauthenticated request when Authorization header is omitted', async ({ apiContext }) => {
    const response = await apiContext.get('wallets/me');
    expect(response.status()).toBe(401);
  });

  test('rejects request with invalid Bearer token', async ({ apiContext }) => {
    const response = await apiContext.get('wallets/me', {
      headers: {
        Authorization: 'Bearer invalid.token.value',
      },
    });
    expect(response.status()).toBe(401);
  });
});
});

```

## Detailed Breakdown of All 8 Test Cases

- **Test 1 (Contract & Zero Balance):** Asserts wallet properties ( `id`, `currency: 'EUR'`, `balance: 0`, `isDefault: true`, `equivalents: object` ) and ensures internal foreign key `userId` is omitted from the DTO.

- **Test 2 (Numeric Precision):** Verifies `Number.isFinite(balance)`, `balance >= 0`, and maximum of 2 decimal places.
- **Test 3 (Cross-Layer DB Parity):** Queries `wallets` table in PostgreSQL and matches `id`, `userId`, `currency`, `balance`, and `isDefault` against the HTTP response.
- **Test 4 (Multi-Currency Equivalents):** Asserts that `equivalents` contains valid finite numbers for all supported currencies: `EUR`, `GBP`, `USD`, `AED`, `PLN`, and `INR`. Confirms base `equivalents['EUR'] === balance`.
- **Test 5 (User Isolation):** Asserts that User A and User B receive distinct wallet IDs and that PostgreSQL assigns them to distinct `userId` foreign keys.
- **Test 6 (Read Idempotence):** Confirms repeated `GET /wallets/me` calls do not mutate state or generate spurious database rows (count remains 1).
- **Test 7 & 8 (Authorization Negatives):** Asserts 401 when Authorization header is omitted or token is invalid.

---

## Part 14 — Cards Test Suite

---

File: `qa/automation/tests/cards/cards.spec.ts`

---

### Purpose

Validates the complete card lifecycle, virtual/physical provisioning, card masking, state transitions ( `active` <-> `frozen` -> `deactivated` ), physical hard deletion, and cross-user IDOR protection.

## Actual Current Code

```
import { test, expect } from '../../../fixtures/api.fixtures';
import { generateTestUserData } from '../../../test-data/user.factory';
import { dbClient } from '../../../database/db-client';
import { CardsApi } from '../../../api';
import { CreateCardRequest } from '../../../api/types';

function generateTestCardData(overrides?: Partial<CreateCardRequest>): CreateCardRequest {
  return {
    cardholderName: overrides?.cardholderName || 'John Doe',
    cardNumber: overrides?.cardNumber || '42424242421234',
    expiryDate: overrides?.expiryDate || '12/28',
    cvv: overrides?.cvv || '123',
    type: overrides?.type,
  };
}

test.describe('Cards Domain API Tests', () => {
  // =====
  // 1. GET /cards - Retrieval & Empty State
  // =====
  test.describe('GET /cards - Listing & Retrieval', () => {
    test('returns an empty array for a newly registered user with no cards', async ({ authUser }) => {
      const response = await authUser.api.cards.getMyCards();
      expect(response.status()).toBe(200);

      const body = await response.json();
      expect(Array.isArray(body)).toBe(true);
      expect(body.length).toBe(0);
    });

    test('returns list of user cards after successful card creation', async ({ authUser }) => {
      const cardPayload = generateTestCardData();
      const createRes = await authUser.api.cards.createCard(cardPayload);
      expect(createRes.status()).toBe(201);
      const createdCard = await createRes.json();

      const listRes = await authUser.api.cards.getMyCards();
      expect(listRes.status()).toBe(200);
      const cards = await listRes.json();

      expect(Array.isArray(cards)).toBe(true);
      expect(cards.length).toBeGreaterThanOrEqual(1);

      const found = cards.find((c: any) => c.id === createdCard.id);
      expect(found).toBeDefined();
      expect(found.cardholderName).toBe('JOHN DOE');
      expect(found.lastFourDigits).toBe('1234');
      expect(found.status).toBe('active');
    });

    test('rejects unauthenticated request when Authorization header is omitted', async ({ apiContext }) => {
      const response = await apiContext.get('cards');
      expect(response.status()).toBe(401);
    });

    test('rejects request with invalid Bearer token', async ({ apiContext }) => {
      const response = await apiContext.get('cards', {
        headers: {
          Authorization: 'Bearer invalid.token.value',
        },
      });
      expect(response.status()).toBe(401);
    });
  });
});

// =====
// 2. POST /cards - Creation & Persistence
// =====
test.describe('POST /cards - Creation & Database Verification', () => {
  test('successfully provisions an active debit card with masked PAN and verifies PostgreSQL persistence', async ({

```

```

    authUser,
  }) => {
    const cardPayload = generateTestCardData({
      cardholderName: 'Alice Springs',
      cardNumber: '5555444433339876',
      expiryDate: '10/29',
      cvv: '999',
    });

    const response = await authUser.api.cards.createCard(cardPayload);
    expect(response.status()).toBe(201);

    const body = await response.json();
    expect(body).toBeDefined();
    expect(typeof body.id).toBe('string');
    expect(body.id.length).toBeGreaterThan(0);
    expect(body.cardholderName).toBe('ALICE SPRINGS');
    expect(body.lastFourDigits).toBe('9876');
    expect(body.expiryDate).toBe('10/29');
    expect(body.status).toBe('active');
    expect(body.type).toBe('debit');

    // Security check: raw card number and CVV must never be returned in API response
    expect(body.cardNumber).toBeUndefined();
    expect(body.cvv).toBeUndefined();

    // Cross-layer PostgreSQL verification
    const dbCard = await dbClient.queryOne<{
      id: string;
      userId: string;
      cardholderName: string;
      lastFourDigits: string;
      expiryDate: string;
      status: string;
      type: string;
    }>(
      'SELECT id, "userId", "cardholderName", "lastFourDigits", "expiryDate", status, type FROM cards WHERE id =
$1',
      [body.id],
    );

    expect(dbCard).not.toBeNull();
    expect(dbCard?.id).toBe(body.id);
    expect(dbCard?.userId).toBe(authUser.user.id);
    expect(dbCard?.cardholderName).toBe('ALICE SPRINGS');
    expect(dbCard?.lastFourDigits).toBe('9876');
    expect(dbCard?.expiryDate).toBe('10/29');
    expect(dbCard?.status).toBe('active');
    expect(dbCard?.type).toBe('debit');
  });

test('successfully provisions a credit card when explicitly requested', async ({ authUser }) => {
  const cardPayload = generateTestCardData({
    type: 'credit',
  });

  const response = await authUser.api.cards.createCard(cardPayload);
  expect(response.status()).toBe(201);
  const body = await response.json();

  expect(body.type).toBe('credit');

  // Cross-layer DB verification
  const dbCard = await dbClient.queryOne<{ type: string }>(
    'SELECT type FROM cards WHERE id = $1',
    [body.id],
  );
  expect(dbCard?.type).toBe('credit');
});

// =====
// 3. POST /cards - Validation Boundaries
// =====

```

```

test.describe('POST /cards - Validation Boundaries', () => {
  test('rejects card creation when cardholderName is missing or empty', async ({ authUser }) => {
    const response = await authUser.api.cards.createCard({
      cardholderName: '',
      cardNumber: '4242424242421234',
      expiryDate: '12/28',
    });
    expect(response.status()).toBe(400);
  });

  test('rejects card creation with invalid expiry date format', async ({ authUser }) => {
    // Month > 12
    const resInvalidMonth = await authUser.api.cards.createCard({
      cardholderName: 'Invalid Expiry',
      cardNumber: '4242424242421234',
      expiryDate: '13/28',
    });
    expect(resInvalidMonth.status()).toBe(400);

    // Non-numeric string
    const resNonNumeric = await authUser.api.cards.createCard({
      cardholderName: 'Invalid Expiry',
      cardNumber: '4242424242421234',
      expiryDate: 'invalid',
    });
    expect(resNonNumeric.status()).toBe(400);
  });

  test('rejects card creation with unsupported card type enum', async ({ authUser }) => {
    const response = await authUser.api.cards.createCard({
      cardholderName: 'Bad Enum',
      cardNumber: '4242424242421234',
      expiryDate: '12/28',
      type: 'prepaid' as any,
    });
    expect(response.status()).toBe(400);
  });
});

// =====
// 4. Card State Machine - Lifecycle
// =====
test.describe('Card State Machine - Lifecycle Transitions', () => {
  test('transitions an active card to frozen status via freeze endpoint', async ({ authUser }) => {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();
    expect(card.status).toBe('active');

    const freezeRes = await authUser.api.cards.freezeCard(card.id);
    expect(freezeRes.status()).toBe(200);
    const frozenCard = await freezeRes.json();
    expect(frozenCard.id).toBe(card.id);
    expect(frozenCard.status).toBe('frozen');

    // Database verification
    const dbCard = await dbClient.queryOne<{ status: string }>('SELECT status FROM cards WHERE id = $1', [card.id]);
    expect(dbCard?.status).toBe('frozen');
  });

  test('transitions a frozen card back to active status via unfreeze endpoint', async ({ authUser }) => {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();
    await authUser.api.cards.freezeCard(card.id);

    const unfreezeRes = await authUser.api.cards.unfreezeCard(card.id);
    expect(unfreezeRes.status()).toBe(200);
    const activeCard = await unfreezeRes.json();
    expect(activeCard.id).toBe(card.id);
    expect(activeCard.status).toBe('active');

    // Database verification
    const dbCard = await dbClient.queryOne<{ status: string }>('SELECT status FROM cards WHERE id = $1', [card.id]);
    expect(dbCard?.status).toBe('active');
  });
});

```

```

    test('transitions an active card to terminal deactivated status via deactivate endpoint', async ({ authUser }) =
> {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();

    const deactivateRes = await authUser.api.cards.deactivateCard(card.id);
    expect(deactivateRes.status()).toBe(200);
    const deactivatedCard = await deactivateRes.json();
    expect(deactivatedCard.id).toBe(card.id);
    expect(deactivatedCard.status).toBe('deactivated');

    // Database verification
    const dbCard = await dbClient.queryOne<{ status: string }>('SELECT status FROM cards WHERE id = $1', [card.i
d]);
    expect(dbCard?.status).toBe('deactivated');
  });

  test('allows deactivating a frozen card directly', async ({ authUser }) => {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();
    await authUser.api.cards.freezeCard(card.id);

    const deactivateRes = await authUser.api.cards.deactivateCard(card.id);
    expect(deactivateRes.status()).toBe(200);
    const deactivatedCard = await deactivateRes.json();
    expect(deactivatedCard.status).toBe('deactivated');

    // Database verification
    const dbCard = await dbClient.queryOne<{ status: string }>('SELECT status FROM cards WHERE id = $1', [card.i
d]);
    expect(dbCard?.status).toBe('deactivated');
  });
});

// =====
// 5. Card State Machine - Invalid Transitions
// =====
test.describe('Card State Machine - Invalid State Transitions', () => {
  test('rejects freezing an already frozen card with 400', async ({ authUser }) => {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();
    await authUser.api.cards.freezeCard(card.id);

    // Second freeze call should be rejected
    const secondFreezeRes = await authUser.api.cards.freezeCard(card.id);
    expect(secondFreezeRes.status()).toBe(400);
    const errorBody = await secondFreezeRes.json();
    expect(errorBody.message).toContain('Card is already frozen');
  });

  test('rejects unfreezing an already active card with 400', async ({ authUser }) => {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();
    expect(card.status).toBe('active');

    const unfreezeRes = await authUser.api.cards.unfreezeCard(card.id);
    expect(unfreezeRes.status()).toBe(400);
    const errorBody = await unfreezeRes.json();
    expect(errorBody.message).toContain('Card is already active');
  });

  test('rejects state modifications on a deactivated card with 400', async ({ authUser }) => {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();
    await authUser.api.cards.deactivateCard(card.id);

    // Attempt freeze on deactivated card
    const freezeRes = await authUser.api.cards.freezeCard(card.id);
    expect(freezeRes.status()).toBe(400);
    const freezeError = await freezeRes.json();
    expect(freezeError.message).toContain('Deactivated card cannot be modified');

    // Attempt unfreeze on deactivated card
    const unfreezeRes = await authUser.api.cards.unfreezeCard(card.id);
    expect(unfreezeRes.status()).toBe(400);
    const unfreezeError = await unfreezeRes.json();
    expect(unfreezeError.message).toContain('Deactivated card cannot be modified');
  });
});

```

```

    // Attempt duplicate deactivation
    const deactivateRes = await authUser.api.cards.deactivateCard(card.id);
    expect(deactivateRes.status()).toBe(400);
    const deactivateError = await deactivateRes.json();
    expect(deactivateError.message).toContain('Card is already deactivated');
  });
});

// =====
// 6. Hard Deletion & IDOR Security Boundaries
// =====
test.describe('Hard Deletion & Cross-User Security', () => {
  test('hard deletes card from PostgreSQL database and excludes it from subsequent listings', async ({
    authUser,
  }) => {
    const card = await (await authUser.api.cards.createCard(generateTestCardData())).json();

    const deleteRes = await authUser.api.cards.deleteCard(card.id);
    expect(deleteRes.status()).toBe(200);
    const deleteBody = await deleteRes.json();
    expect(deleteBody.message).toBe('Card successfully deleted');
    expect(deleteBody.id).toBe(card.id);

    // PostgreSQL verification: card record must be physically deleted (count = 0)
    const dbCount = await dbClient.queryOne<{ count: string }>(
      'SELECT count(*) FROM cards WHERE id = $1',
      [card.id],
    );
    expect(Number(dbCount?.count)).toBe(0);

    // GET /cards must no longer contain the deleted card
    const listRes = await authUser.api.cards.getMyCards();
    const cards = await listRes.json();
    const found = cards.find((c: any) => c.id === card.id);
    expect(found).toBeUndefined();

    // Subsequent delete attempt must return 404
    const secondDeleteRes = await authUser.api.cards.deleteCard(card.id);
    expect(secondDeleteRes.status()).toBe(404);
  });

  test('returns 404 when deleting a non-existent card UUID', async ({ authUser }) => {
    const nonExistentUuid = '00000000-0000-0000-0000-000000000000';
    const response = await authUser.api.cards.deleteCard(nonExistentUuid);
    expect(response.status()).toBe(404);
  });

  test('enforces strict multi-tenant isolation and rejects cross-user card manipulation (IDOR)', async ({
    authUser,
    authApi,
    playwright,
  }) => {
    // User A creates Card A
    const cardA = await (await authUser.api.cards.createCard(generateTestCardData({ cardholderName: 'User A Card'
    }))).json();

    // Provision User B
    const testUserB = generateTestUserData();
    const signupB = await authApi.signup(testUserB);
    expect(signupB.status()).toBe(201);
    await authApi.verifyEmail({ email: testUserB.email, code: '123456' });

    const loginB = await authApi.login({ email: testUserB.email, password: testUserB.password });
    const { access_token: tokenB } = await loginB.json();

    const contextB = await playwright.request.newContext({
      baseURL: (await authUser.api.cards.getMyCards()).url().replace(/\/cards.*$/, '/'),
      extraHTTPHeaders: {
        Authorization: `Bearer ${tokenB}`,
      },
    });

    const cardsApiB = new CardsApi(contextB);

```

```

// User B creates Card B
const cardB = await (await cardsApiB.createCard(generateTestCardData({ cardholderName: 'User B Card' }))).json();

// Multi-tenant visibility isolation: User A sees only Card A, User B sees only Card B
const listA = await (await authUser.api.cards.getMyCards()).json();
const listB = await (await cardsApiB.getMyCards()).json();

expect(listA.some((c: any) => c.id === cardA.id)).toBe(true);
expect(listA.some((c: any) => c.id === cardB.id)).toBe(false);

expect(listB.some((c: any) => c.id === cardB.id)).toBe(true);
expect(listB.some((c: any) => c.id === cardA.id)).toBe(false);

// IDOR attempts: User B attempting to mutate or delete User A's card must return 404
const freezeAttempt = await cardsApiB.freezeCard(cardA.id);
expect(freezeAttempt.status()).toBe(404);

const unfreezeAttempt = await cardsApiB.unfreezeCard(cardA.id);
expect(unfreezeAttempt.status()).toBe(404);

const deactivateAttempt = await cardsApiB.deactivateCard(cardA.id);
expect(deactivateAttempt.status()).toBe(404);

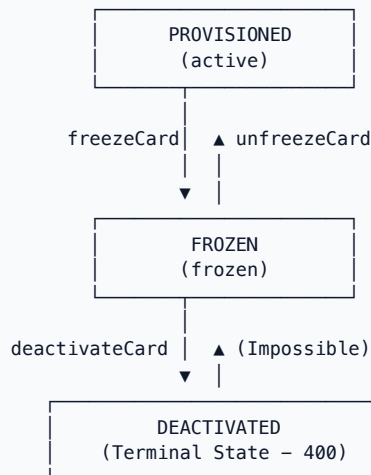
const deleteAttempt = await cardsApiB.deleteCard(cardA.id);
expect(deleteAttempt.status()).toBe(404);

// Confirm Card A status remains untouched in PostgreSQL
const dbCardA = await dbClient.queryOne<{ status: string }>('SELECT status FROM cards WHERE id = $1', [cardA.id]);
expect(dbCardA?.status).toBe('active');

await contextB.dispose();
});
});
});

```

## The Card State Machine



## Detailed Breakdown of Key Cards Tests

- **Test 5 (Debit Provisioning & Masking):** Verifies that sensitive PAN ( `cardNumber` ) and CVV are **never** returned in the API response ( `undefined` ). Only `lastFourDigits` is returned and stored in PostgreSQL.
- **Test 6 (Credit Provisioning):** Proves that explicitly passing `type: 'credit'` is honored and stored.
- **Test 10 & 11 (Active <-> Frozen Transitions):** Tests bidirectional transitions via `/cards/:id/freeze` and `/cards/:id/unfreeze`. Verified in PostgreSQL.
- **Test 12 & 13 (Deactivation Transitions):** Proves that both `active` and `frozen` cards can transition to `deactivated`.

- **Test 14, 15, 16 (Invalid State Transitions):** Confirms state machine guards:
  - Freezing a frozen card returns `400 Bad Request` ( `"Card is already frozen"` ).
  - Unfreezing an active card returns `400 Bad Request` ( `"Card is already active"` ).
  - Attempting any mutation on a `deactivated` card returns `400 Bad Request` ( `"Deactivated card cannot be modified"` ).
- **Test 17 (Hard Deletion):** Calling `DELETE /cards/:id` physically purges the record ( `count = 0` in PostgreSQL). Subsequent delete attempts return `404 Not Found`.
- **Test 19 (IDOR Protection):** Tests Insecure Direct Object References. User B attempts to freeze, unfreeze, deactivate, or delete User A's card. The backend enforces `WHERE id = :id AND userId = :currentUserId` and returns `404 Not Found` for all unauthorized operations. Card A's status remains `'active'` in PostgreSQL.

## Part 15 — Test Data & Test Isolation

### Principles Implemented

1. **Zero Hardcoded IDs or Emails:** Every test suite generates isolated users via `generateTestUserData()`.
2. **Deterministic-Yet-Random:** The generator combines `Date.now()`, `process.pid`, and atomic counters to guarantee zero collisions even across 4+ parallel workers.
3. **Order Independence:** Any test can run in isolation ( `npx playwright test -g "rejects duplicate user"` ), in parallel, or in reverse order without failure.
4. **Lifecycle Scoping:** `APIRequestContext` instances are created per-test and disposed in `fixture.authUser` teardown.
5. **Database Connection Sharing:** All workers share the managed `DbClient` pool which automatically heals if closed.

## Part 16 — Backend Behavior Discovered During Development

### Confirmed from Implementation

- **Port Layout:** Backend runs on `http://localhost:3001/api/v1`; Frontend on `http://localhost:3000`.
- **Signup Contract:** `POST /auth/signup` returns `{ message: string, userId: string }`. It does NOT return an access token or log the user in automatically.
- **Login Contract:** `POST /auth/login` accepts `{ email, password }` and returns `{ access_token: string, user: UserProfile }`.
- **Email Verification:** Verification OTP in development is `'123456'`. Verifying transitions `users.accountStatus` from `'pending'` to `'active'`.
- **Default Wallet:** Signup automatically provisions an initial default wallet in `EUR` with `0.00` balance.
- **Wallet Equivalents:** `GET /wallets/me` calculates real-time equivalents for `['EUR', 'GBP', 'USD', 'AED', 'PLN', 'INR']` using `ExchangeRatesService`. Equivalents are not stored in PostgreSQL.
- **Card State Machine:** Supports `active`, `frozen`, and `deactivated`. `deactivated` is a terminal state.
- **Card Deletion:** `DELETE /cards/:id` is a **hard delete** ( `DELETE FROM cards WHERE id = $1` ).
- **Card Data Masking:** PAN and CVV are never stored in the database. Only `lastFourDigits` is persisted.
- **IDOR Protection:** All card operations filter by `WHERE id = :id AND userId = :userId`, returning 404 if the card belongs to another tenant.
- **BullMQ Architecture:** Queue name is `'transfers'`; job name is `'process-transfer'`; job payload contains `{ transactionId }`.

## Not Implemented / Not Currently Available

- *Direct Wallet Mutations*: There are currently no endpoints for manual deposits or withdrawals ( `POST /wallets/deposit` does not exist).
- *Soft Deletion for Cards*: There is no `deletedAt` column on the cards entity.
- *Server-Side Token Revocation*: `POST /auth/logout` acknowledges logout but does not maintain a Redis JWT revocation blacklist.

# Part 17 — Problems Encountered During Development & Solutions

## Problem 1: Playwright URL Resolution Stripping `/api/v1`

- **What Happened**: Requests to `/auth/login` resulted in `404 Not Found`.
- **Why It Happened**: `new URL('/auth/login', 'http://localhost:3001/api/v1')` treats leading slashes as origin-relative, resolving to `http://localhost:3001/auth/login`.
- **Diagnosis**: Inspected request URLs in Playwright trace viewer.
- **Solution**: Implemented `resolvePath(path)` in `BaseApi`:

```
protected resolvePath(path: string): string {
  return path.startsWith('/') ? path.slice(1) : path;
}
```

- **Why It Works**: Stripping the leading slash forces relative path resolution: `http://localhost:3001/api/v1/` + `auth/login` = `http://localhost:3001/api/v1/auth/login`.

## Problem 2: PostgreSQL Pool Termination in Multi-Worker Execution

- **What Happened**: Tests crashed intermittently with `Error: Cannot use a pool after calling end()`.
- **Why It Happened**: If any hook or worker called `dbClient.close()`, the shared singleton `pg.Pool` instance was marked ended. Subsequent tests using `dbClient` failed.
- **Diagnosis**: Stack trace pointing to `node_modules/pg-pool/index.js`.
- **Solution**: Implemented self-healing pool recreation in `DbClient`:

```
private getActivePool(): Pool {
  if ((this.pool as any).ended) {
    this.initPool();
  }
  return this.pool;
}
```

- **Why It Works**: Transparently detects when the pool was closed and instantiates a fresh pool on the next query.

## Problem 3: BullMQ Blocking Connection Requirement

- **What Happened**: `TransferQueueClient` failed to connect with BullMQ errors.
- **Why It Happened**: BullMQ uses blocking Redis operations ( `BRPOPLPUSH` ). If `maxRetriesPerRequest` is not set to `null`, `ioredis` rejects commands.
- **Diagnosis**: BullMQ documentation and console warnings.
- **Solution**: Configured dedicated IORedis connection:

```
this.connection = new IORedis(url, {
  maxRetriesPerRequest: null,
  lazyConnect: true,
});
```

- **Why It Works:** Satisfies BullMQ protocol requirements.

## Part 18 — Complete Execution Walkthrough

Here is the exact lifecycle of an automated test execution ( `test('transitions an active card to frozen status')` ):

1. **Test Runner Initialization:** Playwright loads `playwright.config.ts`, parses `env.config.ts`, and discovers 64 tests across 7 spec files.
2. **Worker Spawning:** Playwright spawns parallel worker processes.
3. **Fixture Resolution:** Test declares `{ authUser }`.
4. **User Generation:** `generateTestUserData()` creates unique email `qa_1726645800_1234_1@wrightpay-qa.test`.
5. **Signup:** `AuthApi.signup()` sends `POST http://localhost:3001/api/v1/auth/signup`.
6. **Email Verification:** `AuthApi.verifyEmail()` sends OTP `'123456'`. Backend sets `accountStatus: 'active'`.
7. **Login:** `AuthApi.login()` posts credentials and extracts `access_token`.
8. **Context Creation:** Playwright creates an isolated `APIRequestContext` with `Authorization: Bearer <token>`.
9. **Test Body Execution:** Test calls `authUser.api.cards.createCard(cardPayload)`.
10. **Card Provisioning:** Backend creates card in PostgreSQL with status `'active'`.
11. **State Transition:** Test calls `authUser.api.cards.freezeCard(card.id)`.
12. **Status Assertion:** Test asserts response status is `200 OK` and `body.status === 'frozen'`.
13. **Cross-Layer Assertion:** Test invokes `dbClient.queryOne('SELECT status FROM cards WHERE id = $1', [card.id])`. Asserts database row is `'frozen'`.
14. **Teardown:** Playwright runs fixture cleanup, calling `authContext.dispose()`.

## Part 19 — Important SDET Concepts

- **API Testing:** Functional validation of HTTP endpoints, contracts, status codes, and headers.
- **Cross-Layer Testing:** Validating that an API operation produced the correct persistent side effects in PostgreSQL, Redis, and queues.
- **Test Isolation:** Ensuring test cases are 100% independent and can run concurrently without state collision.
- **State Machine Testing:** Verifying valid transitions, terminal states, and asserting that invalid transitions return proper error codes (e.g., 400).
- **IDOR (Insecure Direct Object Reference):** Security testing verifying that Tenant A cannot access or mutate Tenant B's entities.
- **Mass Assignment / Parameter Tampering:** Verifying that API gateways strip non-whitelisted fields (like `id` or `email`) during PATCH/PUT requests.
- **Data Masking (PCI-DSS):** Ensuring sensitive financial attributes (full PAN, CVV, password hashes) are never exposed via API or logged in plaintext.
- **Idempotency:** Guaranteeing that repeated requests produce the exact same outcome without duplicate side effects.

## Part 20 — API Testing vs. Integration Testing

| Dimension                   | Pure API Functional Testing                                                                             | API + DB/Queue Integration Verification                                                                                        |
|-----------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Perspective</b>          | Black-box. Interacts solely with HTTP inputs and outputs.                                               | Grey-box. Interacts with HTTP, then inspects backend storage directly.                                                         |
| <b>Network Target</b>       | Only communicates with the NestJS API gateway.                                                          | Communicates with NestJS API, PostgreSQL, Redis, and BullMQ.                                                                   |
| <b>What It Catches</b>      | Routing errors, validation pipes, HTTP status codes, JSON schema errors.                                | Silent data loss, password hashing bugs, background job failures, cache TTL bugs.                                              |
| <b>Example in WrightPay</b> | Asserting <code>POST /auth/signup</code> returns <code>201 Created</code> and <code>{ userId }</code> . | Asserting <code>users.passwordHash</code> starts with <code>\$argon2</code> and default <code>wallets</code> row exists in DB. |

## Part 21 — SDET Interview Preparation

### Q1: Why did we create API client classes instead of making raw `page.request.post()` calls in tests?

- **Short Answer:** To achieve DRY principles, maintainability, and clean separation of concerns.
- **Deeper Explanation:** If an endpoint path or request schema changes, updating a raw call would require modifying dozens of test files. With API clients, changes are made in one central method. Tests remain readable business specifications.

### Q2: Why don't we put `expect()` assertions inside API clients?

- **Short Answer:** Putting assertions in API clients destroys their reusability for negative testing.
- **Deeper Explanation:** If `createCard()` asserted `expect(response.status()).toBe(201)`, we could never use that method to test invalid inputs, empty names, or unsupported card types that expect `400 Bad Request`. API clients must remain neutral HTTP dispatchers.

### Q3: Why do we use Playwright `APIRequestContext` rather than `axios` or `fetch`?

- **Short Answer:** Unified test framework ecosystem, native browser cookie jar sharing, and zero third-party HTTP dependencies.
- **Deeper Explanation:** `APIRequestContext` is built into `@playwright/test`. It automatically integrates with Playwright's trace viewer, HTML reporters, fixture dependency injection, and network lifecycle management.

### Q4: Why validate PostgreSQL after an API request if the API already returned 200 OK?

- **Short Answer:** To catch silent data corruption, verify data security (Argon2 hashing), and confirm default entity provisioning.
- **Deeper Explanation:** In financial systems, an API might return 200 OK while failing to write to disk, leaving sensitive passwords plaintext, or failing to initialize a default EUR wallet. Cross-layer validation guarantees persistent integrity.

### Q5: How is multi-tenant isolation (IDOR) tested in this framework?

- **Short Answer:** We provision two distinct users in the same test and have User B attempt to access and mutate User A's resources.
- **Deeper Explanation:** In `cards.spec.ts`, User A creates Card A, and User B creates Card B. User B attempts to freeze, unfreeze, deactivate, and delete Card A. We assert that all unauthorized requests return `404 Not Found` and verify via PostgreSQL that Card A remained unchanged.

---

## Part 22 — Debugging Guide

---

### Common Investigatory Scenarios

#### 1. Unexpected 401 Unauthorized:

- Verify JWT expiration.
- Check if endpoint path had a leading slash that stripped `/api/v1`.
- Ensure Authorization header is formatted as `Bearer <token>`.

#### 2. Unexpected 400 Bad Request:

- Inspect backend logs for `ValidationPipe` messages.
- Check DTO constraints in `backend/src/modules/<domain>/dto/`.

#### 3. Database Assertion Mismatch:

- Run manual query using `psql`:

```
docker exec -it wrightpay-postgres psql -U postgres -d wrightpay -c "SELECT * FROM users;"
```

#### 4. Redis Inspection:

- Connect via `redis-cli`:

```
docker exec -it wrightpay-redis redis-cli keys "bull:transfers:"
```

#### 5. Useful Framework Debugging Commands:

- Run with trace viewer: `npx playwright test --trace on`
  - Run in headed UI mode: `npm run test:ui`
  - Open HTML report: `npm run test:report`
  - Run typecheck: `npm run typecheck`
-

## Part 23 — Current Test Inventory

| Domain             | File Path                                | Test Count | What It Covers                             | DB Validation? | Auth Required? | Mutating or Read-Only | Security Rules Tested                           |
|--------------------|------------------------------------------|------------|--------------------------------------------|----------------|----------------|-----------------------|-------------------------------------------------|
| <b>Skeleton</b>    | <code>tests/proof-of-life.spec.ts</code> | 1          | Runner sanity check                        | No             | No             | Read-Only             | None                                            |
| <b>Infra Smoke</b> | <code>tests/infra-smoke.spec.ts</code>   | 3          | PostgreSQL, Redis, BullMQ health           | Yes (Direct)   | No             | Mutating (Ephemeral)  | Connection health                               |
| <b>API Smoke</b>   | <code>tests/api-smoke.spec.ts</code>     | 2          | Public rates & authUser fixture            | No             | Mixed          | Read-Only             | Fixture wiring                                  |
| <b>Auth</b>        | <code>tests/auth/auth.spec.ts</code>     | 19         | Signup, Login, OTP, Logout, JWT boundaries | Yes            | Mixed          | Mutating              | Argon2, JWT forgery, OTP replay, enumeration    |
| <b>Users</b>       | <code>tests/users/users.spec.ts</code>   | 12         | Profile GET/PATCH, IDOR, Whitelisting      | Yes            | Yes            | Mutating              | Mass assignment, isolation                      |
| <b>Wallet</b>      | <code>tests/wallet/wallet.spec.ts</code> | 8          | Schema, Precision, Equivalents, DB parity  | Yes            | Yes            | Read-Only             | Multi-currency invariants, idempotence          |
| <b>Cards</b>       | <code>tests/cards/cards.spec.ts</code>   | 19         | CRUD, State Machine, Hard Delete, IDOR     | Yes            | Yes            | Mutating              | PAN masking, terminal states, cross-tenant IDOR |

**Total Implemented Tests: 64 Passing Tests**

## Part 24 — Complete QA Automation File Index

| File Path                                | Category       | Purpose                                | Major Dependencies                                      | Used By                        |
|------------------------------------------|----------------|----------------------------------------|---------------------------------------------------------|--------------------------------|
| <code>package.json</code>                | Config         | Dependencies & scripts                 | npm                                                     | Framework                      |
| <code>playwright.config.ts</code>        | Config         | Test runner configuration              | <code>@playwright/test</code> , <code>env.config</code> | Test runner                    |
| <code>tsconfig.json</code>               | Config         | TypeScript settings & aliases          | TypeScript                                              | Compiler                       |
| <code>config/env.config.ts</code>        | Config         | Environment loader                     | <code>dotenv</code>                                     | Config, Fixtures, DB, Redis    |
| <code>database/db-client.ts</code>       | Infrastructure | PostgreSQL pool & queries              | <code>pg</code> , <code>env.config</code>               | Fixtures, Smoke & Domain tests |
| <code>redis/redis-client.ts</code>       | Infrastructure | Redis key & TTL client                 | <code>ioredis</code> , <code>env.config</code>          | Fixtures, Smoke tests          |
| <code>queues/queue-client.ts</code>      | Infrastructure | BullMQ queue inspector                 | <code>bullmq</code> , <code>ioredis</code>              | Fixtures, Smoke tests          |
| <code>api/base.api.ts</code>             | API Client     | Abstract HTTP client & pathing         | <code>@playwright/test</code>                           | All domain API clients         |
| <code>api/types.ts</code>                | API Client     | Models, DTOs, & Enums                  | None                                                    | All API clients & tests        |
| <code>api/auth.api.ts</code>             | API Client     | <code>/auth</code> endpoints           | <code>base.api</code> , <code>types</code>              | Fixtures, Auth tests           |
| <code>api/users.api.ts</code>            | API Client     | <code>/users</code> endpoints          | <code>base.api</code> , <code>types</code>              | Fixtures, Users tests          |
| <code>api/wallet.api.ts</code>           | API Client     | <code>/wallets</code> endpoints        | <code>base.api</code>                                   | Fixtures, Wallet tests         |
| <code>api/cards.api.ts</code>            | API Client     | <code>/cards</code> endpoints          | <code>base.api</code> , <code>types</code>              | Fixtures, Cards tests          |
| <code>api/beneficiaries.api.ts</code>    | API Client     | <code>/beneficiaries</code> endpoints  | <code>base.api</code> , <code>types</code>              | Fixtures                       |
| <code>api/transfers.api.ts</code>        | API Client     | <code>/transfers</code> endpoints      | <code>base.api</code> , <code>types</code>              | Fixtures                       |
| <code>api/transactions.api.ts</code>     | API Client     | <code>/transactions</code> endpoints   | <code>base.api</code> , <code>types</code>              | Fixtures                       |
| <code>api/exchange-rates.api.ts</code>   | API Client     | <code>/exchange-rates</code> endpoints | <code>base.api</code> , <code>types</code>              | Fixtures, Smoke tests          |
| <code>api/index.ts</code>                | API Client     | Public barrel export                   | All api files                                           | Fixtures, Tests                |
| <code>test-data/user.factory.ts</code>   | Test Data      | Dynamic user data generator            | <code>types</code>                                      | Fixtures, Domain tests         |
| <code>fixtures/api.fixtures.ts</code>    | Fixtures       | Custom Playwright fixture              | <code>@playwright/test</code> , API clients, DB         | All domain tests               |
| <code>tests/proof-of-life.spec.ts</code> | Tests          | Sanity check                           | <code>@playwright/test</code>                           | Test runner                    |

| File Path                                | Category | Purpose                   | Major Dependencies             | Used By     |
|------------------------------------------|----------|---------------------------|--------------------------------|-------------|
| <code>tests/infra-smoke.spec.ts</code>   | Tests    | Infrastructure health     | DB, Redis, Queue clients       | Test runner |
| <code>tests/api-smoke.spec.ts</code>     | Tests    | API architecture health   | <code>api.fixtures</code>      | Test runner |
| <code>tests/auth/auth.spec.ts</code>     | Tests    | Authentication test suite | <code>api.fixtures</code> , DB | Test runner |
| <code>tests/users/users.spec.ts</code>   | Tests    | Users test suite          | <code>api.fixtures</code> , DB | Test runner |
| <code>tests/wallet/wallet.spec.ts</code> | Tests    | Wallet test suite         | <code>api.fixtures</code> , DB | Test runner |
| <code>tests/cards/cards.spec.ts</code>   | Tests    | Cards test suite          | <code>api.fixtures</code> , DB | Test runner |

## Part 25 — Current Progress

### Completed Steps

- **Step 1:** Repository Audit, Endpoint Inventory, and OpenAPI 3.0 Contract Specification ( `docs/WrightPay-API.yaml` ).
- **Step 2:** QA Automation Foundation, Playwright Configuration, TypeScript compiler, and Proof-of-Life verification.
- **Step 3:** Database ( `DbClient` ), Redis ( `RedisClient` ), and Queue ( `TransferQueueClient` ) infrastructure helpers.
- **Step 4:** Strongly-typed API client architecture, User factory, and pre-authenticated Playwright fixtures ( `authUser` ).
- **Step 5A:** Authentication Domain API Testing (19 tests).
- **Step 5B:** Users Domain API Testing (12 tests).
- **Step 5C:** Wallet Domain API Testing (8 tests).
- **Step 5D:** Cards Domain API Testing (19 tests).
- **Documentation:** Living Master Learning & Reference Guide ( `QA_AUTOMATION_FRAMEWORK_GUIDE.md` ).

**Explicit Status Confirmation:** Beneficiaries testing has **NOT** yet been implemented. No test files have been created in `tests/beneficiaries/` or `tests/api/beneficiaries/`. Work stops here pending user review.

## Part 26 — Future Documentation Sections

The following sections will be appended to this living document as the WrightPay test automation framework expands:

- **Beneficiaries API Testing:** Validation of recipient creation, bank account vs UPI payout routing, validation boundaries, and cross-user deletion.
- **Transfers API Testing:** End-to-end money transfers, idempotency key replay caching, balance debiting, and exchange rate quote conversion.
- **Transactions API Testing:** Transaction ledger retrieval, pagination ( `limit` / `offset` ), status filtering ( `PENDING` , `COMPLETED` , `FAILED` ), and detail queries.
- **Exchange Rates API Testing:** Live rate pair queries, quote calculation accuracy, and inverse rate invariants.
- **Advanced Integration Testing:** Cross-service BullMQ job execution ( `process-transfer` ), async worker verification, and database state transitions.
- **Concurrency & Race Condition Testing:** Simultaneous double-spend transfer attempts and database transaction isolation ( `REPEATABLE READ` / `SERIALIZABLE` ).

- **Frontend / UI Testing:** Next.js Playwright UI automation utilizing Page Object Models ( [pages/](#) ).
- **CI/CD & Reporting:** GitHub Actions workflow automation, artifact collection, and automated test dashboard reporting.